République Tunisienne
Ministère de l'Enseignement Supérieur et
de la Recherche Scientifique

Université de Monastir
FSM

Année Universitaire: 2025-2026

PROJET DE FIN D'ANNÉE

Présenté à

FSM

En vue de l'obtention

De la Licence en GLSI

par

Gritli Ibtihel

CODIFY : APPLICATION WEB INTELLIGENTE ET ASSISTANT IA

Encadrant :

M. Mahmoud Adnen

Tabledesmatières

Liste des tableaux

Table des figures

Introduction générale

Chapitre 1 : Contexte général et étude préalable

1.1 Introduction

1.2 Cadre général

1.3 Présentation de l'organisme d'accueil

1.4 Présentation du projet

1.4.1 Problématique

1.4.2 Étude de l'existant

1.4.3 Critique de l'existant

1.4.4 Solution proposée

1.5 Méthodologie de gestion de projet

1.5.1 Framework SCRUM

1.5.2 Les rôles dans SCRUM

1.5.3 Les artefacts du SCRUM

1.5.4 Les événements du SCRUM

1.6 Conclusion

Chapitre 2 : Analyse et spécification des besoins

2.1 Introduction

2.2 Analyse et spécification des besoins

2.2.1 Identification des acteurs

2.2.2 Besoins fonctionnels

2.2.3 Besoins non fonctionnels

2.2.4 Planification du travail avec Scrum

2.3 Composition du backlog produit

2.3.1 Découpage des Sprints

2.4 Architecture de l'application

2.4.1 Architecture logique (MVC)
2.4.2 Architecture physique (3 tiers)
2.5 Environnement logiciel du projet
2.5.1 Outils de gestion de projet
2.5.2 Frameworks de développement
2.6 Conclusion

Chapitre 3 : Release 1
3.1 Introduction
3.2 Sprint 1 : Gestion des utilisateurs et authentification
3.2.1 Objectif du Sprint 1
3.2.2 Backlog du Sprint 1
3.2.3 Analyse du Sprint 1
3.2.4 Conception du Sprint 1
3.2.5 Réalisation du Sprint 1
3.3 Sprint 2 : Éditeur de code et exécution
3.3.1 Objectif du Sprint 2
3.3.2 Backlog du Sprint 2
3.3.3 Analyse du Sprint 2
3.3.4 Conception du Sprint 2
3.3.5 Réalisation du Sprint 2
3.4 Conclusion

Chapitre 4 : Release 2
4.1 Introduction
4.2 Sprint 3 : Analyse IA et correction de code
4.2.1 Objectif du Sprint 3
4.2.2 Backlog du Sprint 3
4.2.3 Analyse du Sprint 3
4.2.4 Conception du Sprint 3
4.2.5 Réalisation du Sprint 3
4.3 Sprint 4 : Transformation de code et chat IA
4.3.1 Objectif du Sprint 4
4.3.2 Backlog du Sprint 4

4.3.3 Analyse du Sprint 4
4.3.4 Conception du Sprint 4
4.3.5 Réalisation du Sprint 4
4.4 Conclusion

Chapitre 5 : Release 3
5.1 Introduction
5.2 Sprint 5 : Exercices, Hackathons et Gamification
5.2.1 Objectif du Sprint 5
5.2.2 Backlog du Sprint 5
5.2.3 Analyse du Sprint 5
5.2.4 Conception du Sprint 5
5.2.5 Réalisation du Sprint 5
5.3 Panneau Administrateur
5.4 Conclusion

Conclusion générale et Perspectives

Liste des tableaux

1.1 Comparaison entre LeetCode, HackerRank et Codify

2.1 Backlog Produit global

3.1 Backlog Produit du Sprint 1

3.2 Backlog Produit du Sprint 2

4.1 Backlog Produit du Sprint 3

4.2 Backlog Produit du Sprint 4

5.1 Backlog Produit du Sprint 5

5.2 Système de points et gamification

Table des figures

1.2 Logo LeetCode
1.3 Logo HackerRank
1.4 Architecture SCRUM
2.1 Diagramme de cas d'utilisation globale
2.2 Découpage des sprints
2.3 Architecture MVC
2.4 Architecture trois tiers
2.5 Logo GitHub
2.6 Logo Spring Boot
2.7 Logo MySQL
2.8 Logo Thymeleaf
2.9 Logo Bootstrap
2.10 Logo Groq API
3.1 Diagramme de cas d'utilisation du Sprint 1
3.2 Diagramme de classe Sprint 1
3.3 Diagramme de séquence : Authentification
3.4 Diagramme de séquence : Inscription
3.5 Interface page d'accueil
3.6 Interface connexion / inscription
3.7 Interface sélection du niveau
3.8 Diagramme de cas d'utilisation du Sprint 2
3.9 Diagramme de classe Sprint 2
3.10 Diagramme de séquence : Exécution du code
3.11 Interface éditeur de code
3.12 Interface résultat d'exécution
4.1 Diagramme de cas d'utilisation du Sprint 3
4.2 Diagramme de classe Sprint 3
4.3 Diagramme de séquence : Analyse IA
4.4 Interface panneau IA — Analyse
4.5 Interface panneau IA — Optimisation
4.6 Diagramme de cas d'utilisation du Sprint 4

4.7 Diagramme de classe Sprint 4
4.8 Diagramme de séquence : Transformation de code
4.9 Interface transformation Python → Java
4.10 Interface chat IA
5.1 Diagramme de cas d'utilisation du Sprint 5
5.2 Diagramme de classe Sprint 5
5.3 Diagramme de séquence : Résolution d'exercice
5.4 Diagramme de séquence : Hackathon temps réel
5.5 Interface exercices adaptés par niveau
5.6 Interface modal exercice avec timer
5.7 Interface hackathon — arène
5.8 Interface résultat final hackathon
5.9 Interface dashboard utilisateur
5.10 Interface classement par niveau
5.11 Interface panneau administrateur
5.12 Diagramme de classe globale

Introduction générale :

-Dans un monde où la technologie évolue à une vitesse croissante, la programmation est devenue une compétence fondamentale recherchée dans tous les secteurs d'activité. Pourtant, apprendre à programmer reste un défi difficile pour de nombreux étudiants et autodidactes. Les erreurs de syntaxe, la compréhension des algorithmes, le choix du langage adapté et la recherche de solutions optimisées constituent autant d'obstacles qui freinent la progression des apprenants.

Face à ce constat, il est devenu évident que la solution ne réside pas dans des cours théoriques statiques, mais dans une approche interactive, personnalisée et immédiate. Un apprenant progresse plus rapidement lorsqu'il comprend ses erreurs au moment où il les commet, lorsqu'il reçoit des explications adaptées à son niveau de compréhension, et lorsqu'il est encouragé à penser de manière optimale dès le début de son apprentissage.

C'est dans ce contexte que le projet Codify a été conçu. Codify est une plateforme web intelligente d'apprentissage de la programmation assistée par intelligence artificielle. Elle accompagne l'apprenant à chaque étape de son parcours : elle détecte ses erreurs et les explique clairement, elle analyse son code et propose des versions corrigées et optimisées, elle l'aide à comprendre la différence entre les langages de programmation, et elle mesure sa complexité algorithmique pour l'encourager à penser de manière plus efficace.

Codify offre également un système d'exercices pratiques générés automatiquement selon le niveau de l'utilisateur, inspirés de vraies plateformes de programmation compétitive. Des hackathons en temps réel permettent aux apprenants de se mesurer à d'autres utilisateurs sur des problèmes algorithmiques réels, rendant l'apprentissage à la fois stimulant et ludique. Un système de points, de classement et de progression de niveau motive l'utilisateur à s'améliorer continuellement.

Notre rapport est structuré en cinq chapitres principaux :

Le premier chapitre, « Contexte général et étude préalable », présente le cadre général du projet, la problématique identifiée, l'étude des solutions existantes et leurs limites, ainsi que la méthodologie SCRUM adoptée.

Le deuxième chapitre, « Analyse et spécification des besoins », identifie les acteurs du système, décrit les besoins fonctionnels et non fonctionnels, présente le backlog produit et le

découpage en sprints, et décrit l'architecture de l'application ainsi que l'environnement logiciel utilisé.

Le troisième chapitre, « Release 1 », traite des deux premiers sprints : la gestion des utilisateurs et l'authentification d'une part, et l'éditeur de code avec l'exécution d'autre part.

Le quatrième chapitre, « Release 2 », aborde le Sprint 3 consacré à l'analyse intelligente du code par l'IA, et le Sprint 4 dédié à la transformation de code et au chat IA.

Le cinquième chapitre, « Release 3 », présente le Sprint 5 qui regroupe le système d'exercices adaptatifs, les hackathons en temps réel et le panneau d'administration.

Chapitre 1

Contexte général et étude préalable

1.1 Introduction :

-Le premier chapitre de ce rapport se concentre sur le contexte général et l'étude préalable du projet Codify. Nous commencerons par présenter le cadre général du projet, suivi d'une description détaillée de la problématique à résoudre. Nous examinerons ensuite les solutions existantes sur le marché, leurs points forts et leurs limites. Enfin, nous présenterons la solution proposée et la méthodologie de gestion de projet adoptée, en mettant l'accent sur le framework SCRUM et ses différents éléments.

1.2 Cadre général :

-Le présent projet **Codify** est réalisé dans le cadre d'un projet de fin d'année pour l'obtention du diplôme de Licence en Génie Logiciel et Systèmes Informatiques (GLSI), associé à l'année universitaire 2025-2026. Il a été réalisé au sein de la Faculté des Sciences de Monastir (FSM).

1.4 Présentation du projet :

1.4.1 Problématique :

-L'apprentissage de la programmation est une activité intellectuelle exigeante qui demande de la rigueur, de la logique et une capacité à comprendre rapidement ses erreurs. Or, les méthodes d'enseignement traditionnelles ne permettent pas toujours à l'étudiant de recevoir un retour immédiat et personnalisé sur ses erreurs. Un étudiant qui écrit du code incorrect doit souvent attendre la correction d'un enseignant ou parcourir de longues documentations pour comprendre son problème, ce qui ralentit considérablement sa progression.
Par ailleurs, les plateformes de programmation existantes telles que LeetCode ou HackerRank sont conçues pour des programmeurs déjà expérimentés. Elles retournent un simple résultat de réussite ou d'échec sans expliquer la cause de l'erreur ni proposer de piste d'amélioration. Un étudiant débutant qui soumet une solution incorrecte ne comprend pas pourquoi son code est faux ni comment l'améliorer.

De plus, la notion de complexité algorithmique, qui est fondamentale pour écrire des programmes efficaces, est rarement enseignée de manière pratique et progressive. Les apprenants ont tendance à écrire des solutions qui fonctionnent mais qui sont inefficaces, sans jamais être guidés vers des approches plus optimales.

On peut donc formuler la problématique centrale du projet comme suit :

« Comment aider un apprenant en programmation à comprendre ses erreurs, à améliorer son niveau de manière progressive, à développer une pensée algorithmique optimisée et à se motiver grâce à des défis ludiques, le tout adapté à son niveau personnel ? »

1.4.2 Étude de l'existant :

-Afin de proposer une solution pertinente, nous avons analysé les deux plateformes les plus représentatives dans le domaine de l'apprentissage de la programmation :

LeetCode est une plateforme américaine très populaire proposant plus de 3000 problèmes algorithmiques classés par difficulté. Elle est principalement utilisée pour préparer les entretiens techniques dans les grandes entreprises technologiques comme Google, Amazon ou Meta. Elle propose un éditeur de code en ligne, un système de soumission automatique, des statistiques de performance et un classement mondial.



Figure 1.2– Logo de LeetCode

HackerRank est une plateforme de programmation compétitive qui propose des défis dans de nombreux langages de programmation et dans plusieurs domaines tels que les algorithmes, les structures de données, les mathématiques et l'intelligence artificielle. Elle est utilisée par des entreprises pour recruter des développeurs et par des étudiants pour s'entraîner.



Figure 1.3– Logo de HackerRank

1.4.3 Critique de l'existant :

L'analyse des solutions existantes nous a permis d'identifier plusieurs limites importantes qui justifient la création d'une nouvelle plateforme :

Fonctionnalité	LeetCode	HackerRank	Codify

Exécution de code dans le navigateur	✓ **(limitée)**	✓ **(limitée)**	✓ **Complète**
Correction des erreurs par IA	✗ **(pas d'IA)**	✗ **(pas d'IA)**	✓ **API**
Explications adaptées au niveau	✗	✗	✓ **3 niveaux**
Transformation inter-langages	✗	✗	✓ **(Python, Java, C)**
Exercices personnalisés par niveau	✓	✓	✓ **Adaptés par IA**
Gamification et classement	✓	✓	✓ **+ Récompenses**
Défis en équipe	✗	✓ **(limité)**	✓
Recommandation de langage par IA	✗	✗	✓

Tableau 1.1 — Comparaison entre LeetCode et HackerRank

-La limite principale commune à ces deux plateformes est l'absence totale d'une intelligence artificielle pédagogique qui s'adapte au niveau de l'utilisateur. Elles ne proposent aucune explication des erreurs, aucun conseil personnalisé et aucune analyse de la qualité du code soumis. Pour un étudiant débutant, ces plateformes sont souvent décourageantes car elles ne l'accompagnent pas dans sa progression.

1.4.4 Solution proposée :

-Pour résoudre se problème nous avons créé Codify, une plateforme en ligne qui utilise l'intelligence artificielle pour aider à apprendre la programmation. Codify se distingue des solutions existantes par une approche entièrement centrée sur l'apprenant et sa progression personnelle.

La plateforme accompagne l'utilisateur à chaque étape de son apprentissage. À l'inscription, il choisit son niveau parmi trois options (Débutant, Intermédiaire ou Avancé) et toutes les fonctionnalités s'adaptent automatiquement à ce niveau. Lorsqu'il écrit du code et l'exécute, l'intelligence artificielle analyse immédiatement son code et retourne une réponse structurée contenant les erreurs détectées avec leur numéro de ligne exact, le code corrigé commenté, une méthode alternative plus simple ou plus optimale, et des conseils personnalisés adaptés à son niveau.

Codify intègre également une fonctionnalité de transformation de code qui permet de convertir un programme d'un langage vers un autre (Python, Java, C) avec des explications des différences de syntaxe, de logique et de performance. Un chat IA interactif permet à

l'utilisateur de poser toutes ses questions sur son code et d'obtenir des réponses contextualisées.
Pour la partie gamifiée, Codify propose des exercices de programmation générés automatiquement par l'IA selon le niveau et le langage choisis, inspirés de vraies plateformes comme LeetCode et Codeforces. Chaque exercice est accompagné d'un timer pour mesurer le temps de résolution et d'une analyse de la complexité algorithmique de la solution. Un système de points avec bonus de rapidité motive l'utilisateur à progresser. Après 20 exercices, un test de validation chronométré permet à l'utilisateur de changer de niveau s'il le réussit. Enfin, des hackathons en temps réel permettent à deux utilisateurs de s'affronter sur un même problème algorithmique, avec des timers parfaitement synchronisés et une évaluation comparative des deux solutions par l'IA à la fin de chaque hackathon

1.5 Méthodologie de gestion de projet à adopter :

-Bien évidemment, il existe plusieurs méthodologies de gestion de projet chacune avec ses avantages et ses inconvénients. Ici, nous avons choisi le Framework SCRUM vu sa flexibilité et vu qu'elle met l'accent sur la qualité du produit fini.

1.5.1 Framework SCRUM :

-Pour la gestion de ce projet, nous avons adopté la méthodologie SCRUM, un framework agile qui permet de livrer des fonctionnalités de manière incrémentale et itérative. SCRUM divise le projet en cycles de développement courts appelés Sprints, chacun produisant une version fonctionnelle et testable du logiciel.

SCRUM a été choisi pour ce projet pour plusieurs raisons. Premièrement, il permet d'adapter le développement aux retours obtenus après chaque sprint, ce qui est particulièrement utile lorsque les besoins évoluent en cours de projet. Deuxièmement, il favorise une livraison régulière de fonctionnalités utilisables, ce qui permet de tester et de valider chaque partie du système progressivement. Troisièmement, il offre une grande visibilité sur l'avancement du projet grâce aux différents artefacts et événements qu'il définit.

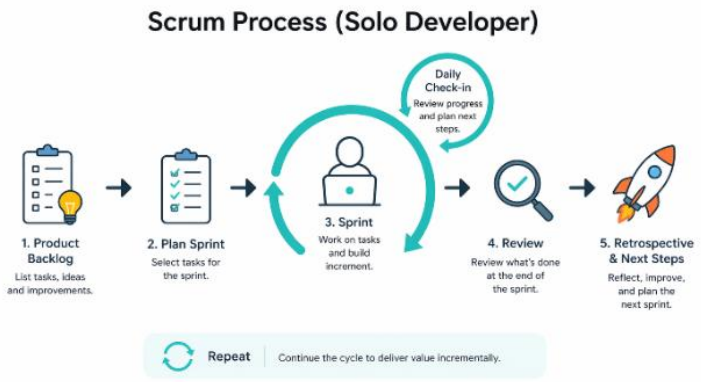


Figure 1.4 l'architecture générale du framework SCRUM

1.5.2 Les rôles dans SCRUM :

Le framework SCRUM définit deux rôles principaux dont les responsabilités sont clairement délimitées :

Scrum Master : Il est responsable de veiller à l'application correcte du framework SCRUM et de lever les obstacles qui bloquent l'équipe. Dans mon projet, ce rôle est assuré par moi

Équipe de développement : Elle est responsable de la réalisation des fonctionnalités définies dans le sprint backlog. Dans mon projet, c'est mois qui assure à la fois le développement backend (Spring Boot), le développement frontend (Thymeleaf, Bootstrap, JavaScript) et l'intégration de l'intelligence artificielle.

1.5.3 Les artefacts du SCRUM :

Le framework SCRUM définit trois artefacts principaux qui assurent la transparence du projet:

Product Backlog : Liste ordonnée par priorité de toutes les fonctionnalités souhaitées pour le produit final. Dans notre projet, il contient 21 user stories réparties sur 5 sprints, allant de la gestion des utilisateurs jusqu'aux hackathons en temps réel.

Sprint Backlog : Sous-ensemble du Product Backlog sélectionné pour être réalisé durant un sprint donné. Il contient les tâches détaillées nécessaires à la réalisation des user stories sélectionnées pour ce sprint.

Incrément : Version fonctionnelle et testable du logiciel produite à la fin de chaque sprint. Chaque incrément s'appuie sur les incréments précédents et ajoute de nouvelles fonctionnalités. À la fin du Sprint 5, l'incrément final constitue la version complète de Codify.

1.6 Conclusion :

-Ce premier chapitre nous a permis de poser les bases solides du projet Codify. Nous avons présenté le cadre général du projet, défini la problématique centrale et analysé les limites des solutions existantes sur le marché. Nous avons ensuite proposé Codify comme une solution innovante basée sur l'intelligence artificielle et la gamification pour accompagner les apprenants dans leur progression en programmation. Enfin, nous avons décrit la méthodologie SCRUM adoptée pour structurer le développement de manière agile et itérative. Le chapitre suivant présentera l'analyse détaillée des besoins, l'architecture de l'application et l'environnement logiciel utilisé.

Chapitre 2

Analyse et spécification des besoins

2.1 Introduction :

-Le deuxième chapitre est consacré à l'analyse et à la spécification des besoins de la plateforme Codify. Nous commençons par identifier les acteurs du système, puis nous décrivons les besoins fonctionnels et non fonctionnels. Nous présentons ensuite la planification du travail avec SCRUM à travers le backlog produit et le découpage en sprints. Enfin, nous décrivons l'architecture logique et physique de l'application ainsi que l'environnement logiciel et les technologies utilisées.

2.2 Analyse et spécification des besoins :

-Tout au long de cette section, nous allons identifier et spécifier les besoins à satisfaire, qui représentent les fonctionnalités à mettre en œuvre dans notre application

2.2.1 Identification des acteurs

-Le système Codify implique deux acteurs principaux dont les rôles et les accès sont bien distincts :

L'Utilisateur : Tout étudiant et tout personne passionne par la programmation et le codage ou qui va améliorer son niveau de programmation et apprendre de nouvelle technique ou apprenant qui s'inscrit sur la plateforme. Il choisit son niveau à l'inscription (Débutant, Intermédiaire ou Avancé) et accède à toutes les fonctionnalités pédagogiques de Codify : éditeur de code avec analyse IA, transformation de code, chat IA, exercices adaptatifs, test de niveau, hackathons en temps réel et tableau de bord personnalisé.

L'Administrateur : Gestionnaire unique de la plateforme. Il accède à un espace d'administration sécurisé par session côté serveur, lui permettant de consulter les trois tableaux de classement, de voir le bilan complet des hackathons et de gérer les comptes des utilisateurs inscrits.

-La figure 2.1 présente le diagramme de cas d'utilisation globale de notre future application web :

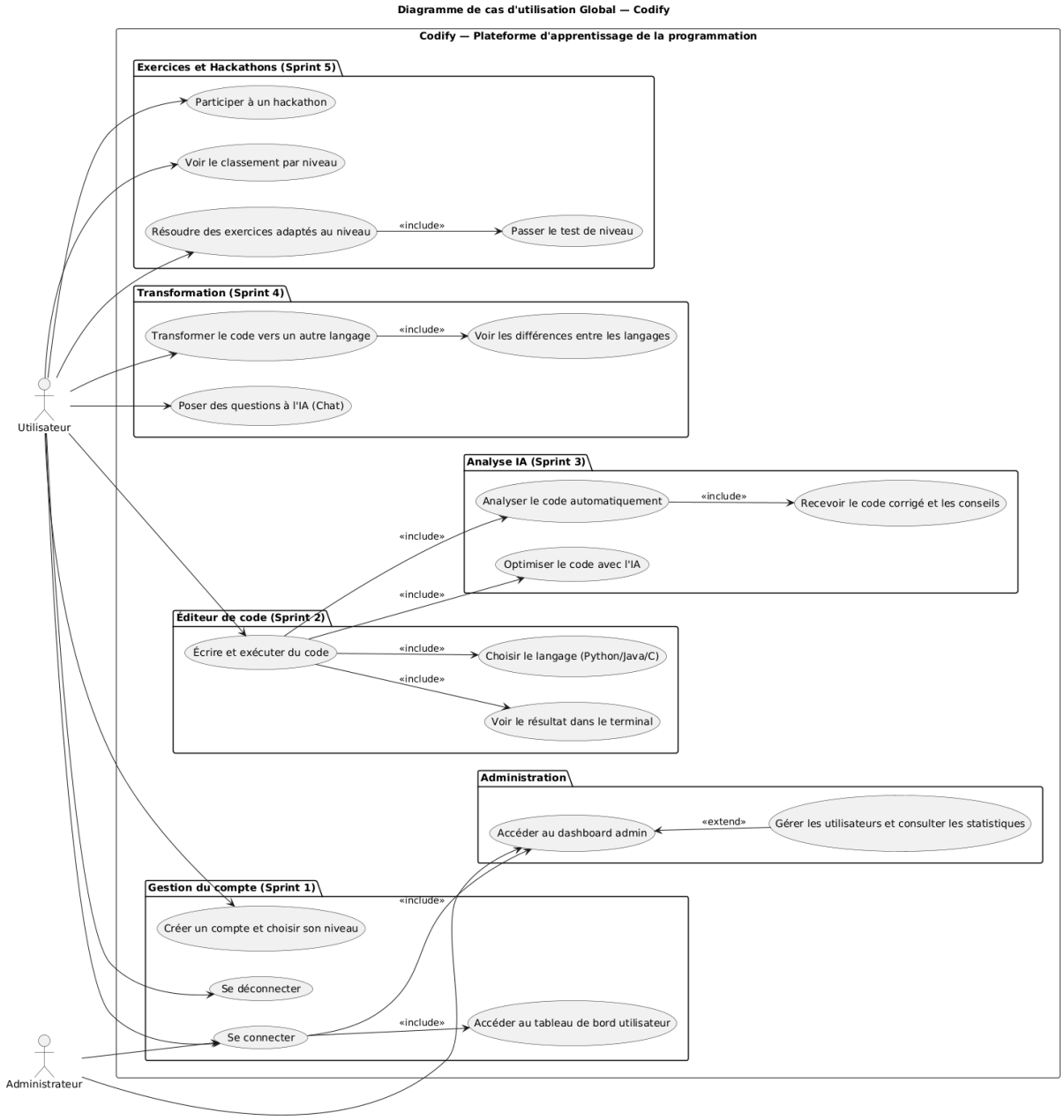


Figure 2.1 présente le diagramme de cas d'utilisation global

-La figure 2.2 présente le diagramme de classe globale de notre future application web :

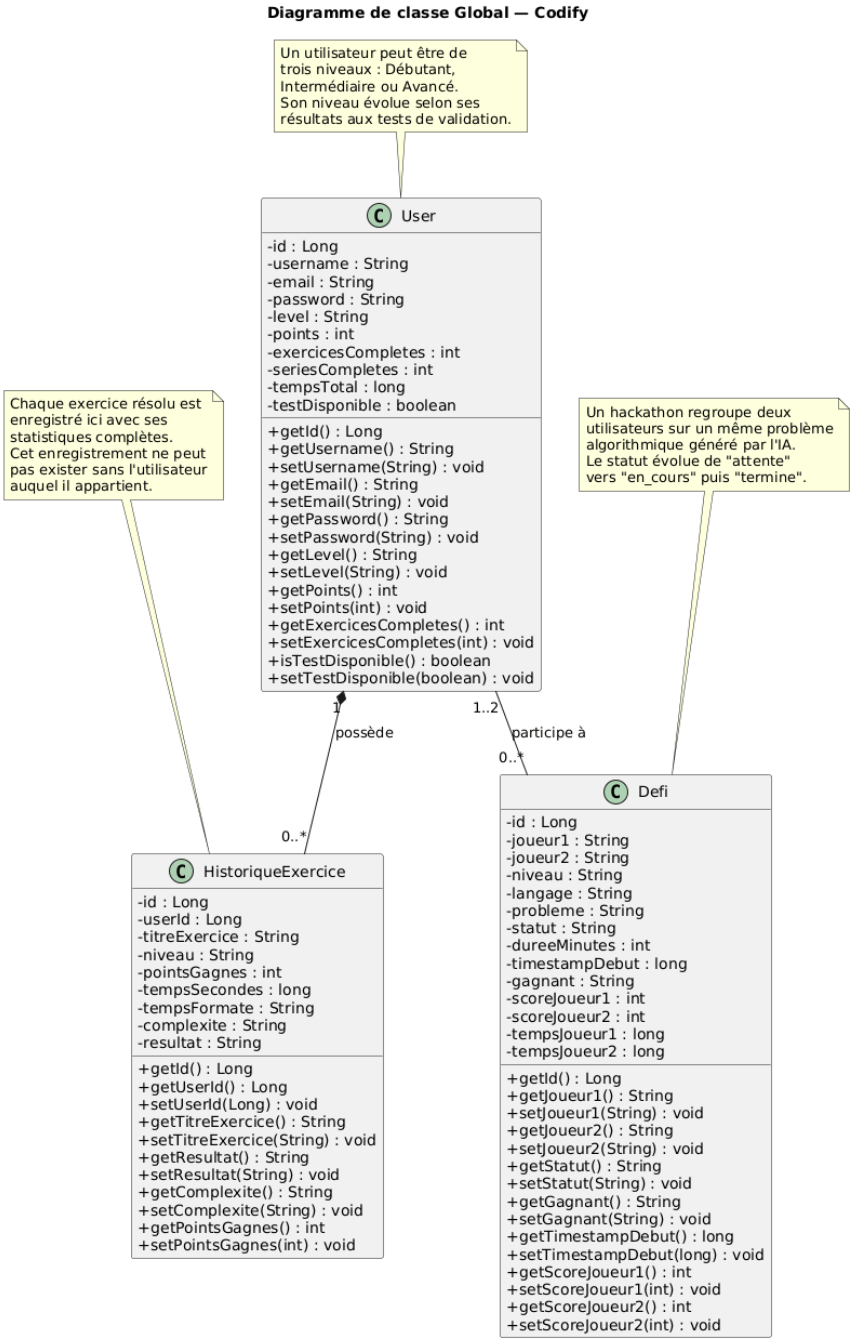


Figure 2.2 présente le diagramme de classe globale

Classe User : Représente un utilisateur de la plateforme

Classe HistoriqueExercice : Enregistre chaque exercice résolu par un utilisateur.

Classe Defi : Représente un hackathon (défi) entre deux utilisateurs**.**

2.2.2 Besoins fonctionnels :

-Les besoins fonctionnels décrivent les fonctionnalités que le système doit offrir à ses utilisateurs. Ils sont organisés par sprint :

Sprint 1 : Gestion des utilisateurs :

- L'utilisateur doit pouvoir créer un compte en fournissant son nom, son email et son mot de passe, et en choisissant son niveau.
- L'utilisateur doit pouvoir se connecter avec son email et son mot de passe.
- L'utilisateur doit accéder à un tableau de bord affichant ses points, son niveau, ses exercices complétés et son classement.
- Le tableau de bord doit afficher l'historique complet des exercices avec le titre, le niveau, les points gagnés, le temps, la complexité et le résultat.
- L'administrateur doit pouvoir consulter les classements par niveau et gérer les comptes utilisateurs.

Sprint 2 : Éditeur de code :

- L'utilisateur doit pouvoir écrire du code dans un éditeur intégré.
- Le système doit prendre en charge trois langages : Python, Java et C.
- Le système doit exécuter le code et afficher le résultat dans un terminal intégré.
- Pour Python, le système doit détecter les instructions d'affichage et retourner leurs valeurs.

Sprint 3 :Analyse intelligente du code :

- Le système doit analyser automatiquement le code après chaque exécution.
- Les règles locales programmées doivent détecter les erreurs simples et retourner un message adapté au niveau.
- L'IA externe doit analyser le code en profondeur et retourner une réponse en 5 sections : explication, erreurs, code corrigé, méthode alternative et conseils.
- L'IA doit proposer automatiquement une version optimisée du code.

Sprint 4 : Transformation et Chat IA :

- L'utilisateur doit pouvoir transformer son code d'un langage source vers un langage cible différent.
- L'IA doit expliquer les différences de syntaxe, de logique et de performance entre les deux langages.
- L'utilisateur doit pouvoir poser des questions libres à l'IA sur son code dans un chat interactif.

Sprint 5 :Exercices et Hackathons :

- Le système doit générer 6 exercices de programmation adaptés au niveau et au langage choisis.
- Chaque exercice doit être accompagné d'un timer pour mesurer le temps de résolution.
- La correction doit attribuer des points selon la qualité (correct, partiel, incorrect) avec un bonus de rapidité.
- La complexité algorithmique de chaque solution doit être extraite et affichée.
- L'utilisateur doit pouvoir passer un test de validation chronométré après 20 exercices complétés.
- Le test réussi doit permettre à l'utilisateur de passer au niveau supérieur.
- L'utilisateur doit pouvoir créer et rejoindre des hackathons en temps réel avec timer synchronisé.
- L'IA doit comparer les deux solutions à la fin de chaque hackathon.

2.2.3 Besoins non fonctionnels :

Les besoins non fonctionnels définissent les contraintes de qualité que le système doit respecter indépendamment des fonctionnalités offertes :

Performance : L'analyse IA doit retourner une réponse en moins de 3 secondes. Le modèle Groq llama-3.3-70b-versatile est optimisé pour la rapidité d'inférence, ce qui garantit une expérience fluide pour l'utilisateur.

Sécurité : Les sessions utilisateurs sont gérées côté serveur par Spring Boot. L'accès à toutes les pages protégées est conditionné par l'existence d'une session valide. L'espace administrateur est sécurisé par des identifiants distincts stockés sous forme de constantes côté serveur.

Adaptabilité : L'interface et les explications de l'IA s'adaptent automatiquement au niveau de chaque utilisateur sans intervention manuelle. Les prompts envoyés à l'IA sont construits dynamiquement selon le niveau.

Temps réel : Les hackathons doivent fonctionner en temps réel avec une communication instantanée entre les deux joueurs grâce à la technologie WebSocket STOMP sur SockJS. La synchronisation des timers est assurée par un timestamp serveur unique transmis aux deux joueurs.

Maintenabilité : L'architecture en couches MVC (Modèle-Vue-Contrôleur) facilite les modifications et les ajouts de fonctionnalités. Chaque couche est indépendante et testable séparément.

Utilisabilité : L'interface est conçue avec un thème sombre professionnel, des retours visuels clairs sous forme de cartes colorées pour les résultats IA, des barres de

progression pour l'avancement, et des timers visibles pour les exercices et les hackathons.

2.2.4 Planification du travail avec Scrum :

-Le projet Codify est organisé en 3 Releases regroupant 5 Sprints de durées variables selon la complexité des fonctionnalités à développer. Cette organisation permet une livraison progressive et vérifiable du système :

Release	**Sprints**	**Fonctionnalités principales**
Release 1	Sprint 1 + Sprint 2	Gestion des utilisateurs et éditeur de code
Release 2	Sprint 3 + Sprint 4	Analyse IA et transformation de code
Release 3	Sprint 5	Exercices, hackathons et gamification

Tableau 2.1 — Organisation des Releases et des Sprints

La figure 2.2 présente le découpage des sprints dans le temps et la répartition des fonctionnalités par release.

2.3 Composition du backlog produit :

-Le tableau suivant présente l'ensemble des user stories du backlog produit de Codify, ordonnées par priorité et réparties par sprint. Chaque user story est formulée du point de vue de l'utilisateur selon le format standard de SCRUM : « En tant que [acteur], je veux [fonctionnalité] pour [bénéfice] ».

ID	**Fonctionnalité**	**User Story**	**Priorité**	**Complexité**
1.1	Inscription	En tant qu'utilisateur, je veux créer un compte pour accéder aux fonctionnalités.	Haute	Faible
1.2	Authentification	En tant qu'utilisateur, je veux me connecter pour sauvegarder mon activité.	Haute	Faible

1.3	Choix du niveau	En tant qu'utilisateur, je veux choisir mon niveau pour que l'IA adapte ses explications.	Haute	Moyenne
1.4	Gestion Admin	En tant qu'administrateur, je veux gérer les comptes et voir les statistiques.	Moyenne	Moyenne
2.1	Éditeur de code	En tant qu'utilisateur, je veux écrire du code en différents langages.	Haute	Moyenne
2.2	Multi-langage	En tant qu'utilisateur, je veux choisir entre Python, Java et C.	Haute	Élevée
2.3	Exécution	En tant qu'utilisateur, je veux exécuter mon code et voir le résultat.	Haute	Moyenne
2.4	Affichage erreurs	En tant qu'utilisateur, je veux voir les erreurs si mon code ne fonctionne pas.	Haute	Moyenne
3.2	Assistant IA externe	En tant qu'utilisateur, je veux que l'IA explique mes erreurs selon mon niveau.	Haute	Élevée
3.3	Correction	En tant qu'utilisateur, je veux recevoir une proposition de code corrigé.	Haute	Élevée
3.4	Optimisation	En tant qu'utilisateur, je veux que l'IA propose une version optimisée.	Moyenne	Élevée
4.1	Transformation	En tant qu'utilisateur, je veux transformer mon code vers un autre langage.	Haute	Élevée
4.2	Différences langages	En tant qu'utilisateur, je veux comprendre les différences entre les langages.	Haute	Moyenne

4.3	Chat IA	En tant qu'utilisateur, je veux poser des questions libres à l'IA sur mon code.	Moyenne	Moyenne
5.1	Exercices IA	En tant qu'utilisateur, je veux résoudre des exercices adaptés à mon niveau.	Haute	Moyenne
5.2	Test de niveau	En tant qu'utilisateur, je veux passer un test pour vérifier ma progression.	Haute	Moyenne
5.3	Mise à jour niveau	En tant qu'utilisateur, je veux que mon niveau soit mis à jour après le test.	Haute	Faible
5.4	Dashboard enrichi	En tant qu'utilisateur, je veux voir mes exercices, le temps et le classement.	Moyenne	Moyenne
5.5	Hackathons	En tant qu'utilisateur, je veux participer à des défis en temps réel.	Moyenne	Élevée
5.6	Récompenses	En tant qu'utilisateur, je veux recevoir des points selon mes performances.	Moyenne	Faible

Tableau 2.2 — Backlog Produit de Codify

2.3.1 Découpage des Sprints :

-La figure 2.4 présente le découpage des sprints dans le temps. Le tableau suivant résume les objectifs et le contenu de chaque sprint :

Sprint	Objectif	User Stories	Release
Sprint 1	Gestion des utilisateurs et authentification	1.1, 1.2, 1.3, 1.4	Release 1
Sprint 2	Éditeur de code et exécution	2.1, 2.2, 2.3, 2.4	Release 1

Sprint 3	Analyse intelligente du code par l'IA	3.1, 3.2, 3.3, 3.4	Release 2
Sprint 4	Transformation de code et Chat IA	4.1, 4.2, 4.3	Release 2
Sprint 5	Exercices, hackathons et gamification	5.1, 5.2, 5.3, 5.4, 5.5, 5.6	Release 3

Tableau 2.3 — Découpage des Sprints

2.4 Architecture de l'application :

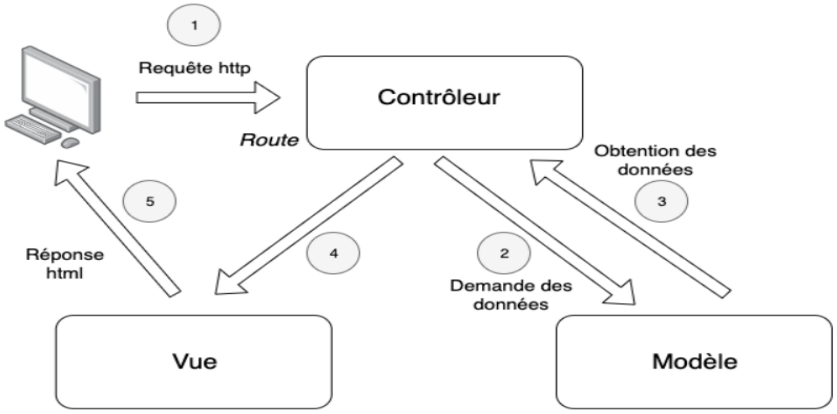
2.4.1 Architecture logique :

-Codify repose sur une architecture MVC (Modèle-Vue-Contrôleur) à trois couches bien séparées, ce qui garantit la maintenabilité et l'évolutivité du code :

La couche Modèle : Elle contient les entités Java annotées avec JPA (User, HistoriqueExercice, Defi) qui représentent les tables de la base de données MySQL. Elle contient également les services qui encapsulent toute la logique métier de l'application et les repositories qui assurent la communication avec la base de données.

La couche Vue : Elle contient les pages HTML développées avec Thymeleaf et Bootstrap 5, ainsi que le code JavaScript (ide.js) qui gère les interactions de l'utilisateur et les appels aux APIs du serveur.

La couche Contrôleur : Elle contient les contrôleurs Spring Boot qui reçoivent les requêtes HTTP, appellent les services appropriés et retournent soit des pages HTML (via Thymeleaf), soit des réponses en format JSON (via les annotations @ResponseBody pour les APIs REST).



La figure 2.3 présente l'architecture MVC appliquée à Codify.

2.4.2 Architecture physique :

-L'architecture physique de Codify repose sur trois niveaux distincts qui communiquent entre eux selon un flux bien défini :

Niveau Client : Le navigateur web de l'utilisateur qui affiche les pages HTML et exécute le code JavaScript. La communication avec le serveur se fait via des requêtes HTTP standard pour les fonctionnalités classiques et via des connexions WebSocket STOMP pour les hackathons en temps réel.

Niveau Serveur (Spring Boot) : Le serveur Spring Boot qui tourne localement sur le port 9090. Il traite toutes les requêtes HTTP, gère les sessions utilisateurs, construit les prompts adaptés au niveau de l'utilisateur, appelle l'API Groq externe pour l'analyse IA, et gère les connexions WebSocket pour les hackathons.

Niveau Données (MySQL) : La base de données MySQL nommée codify_db qui stocke les trois tables du système : users (comptes utilisateurs), historique_exercices (résultats des exercices) et defis (hackathons). La communication avec MySQL est assurée par Spring Data JPA.

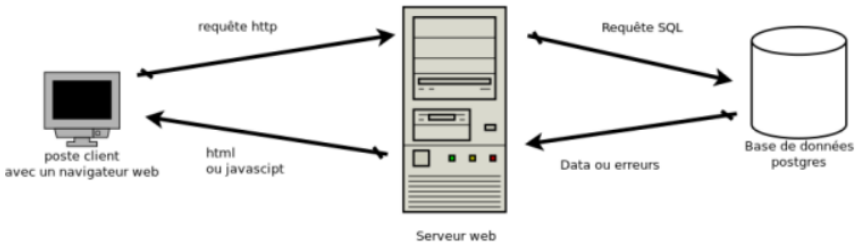


Figure 2.4 présente l'architecture physique à trois niveaux

La figure 2.4 présente l'architecture physique à trois niveaux de Codify ainsi que les flux de communication entre les différents composants.

2.5 Environnement logiciel du projet :

-Dans cette partie, nous allons présenter les outils de gestion de projet et les frameworks de développement et de tests utilisés pour réaliser Codify.

2.5.1 Outils de gestion de projet :

Outil	**Rôle dans le projet**
GitHub	Gestion du code source et versioning. Permet de sauvegarder le code et de suivre l'historique des modifications à chaque sprint.

Spring Tools Suite 4 (STS)	Environnement de développement intégré spécialement conçu pour les projets Spring Boot. Utilisé pour développer, compiler et démarrer le serveur backend sur le port 9090.
MySQL Workbench	Outil de gestion et de visualisation de la base de données MySQL. Utilisé pour inspecter les tables, vérifier les données enregistrées et tester les requêtes.
Postman	Outil de test des APIs REST. Utilisé pour tester les routes du backend avant de les connecter au frontend JavaScript.

Tableau 2.4 — Outils de gestion de projet

2.5.2 Frameworks de développement et de tests :

Technologie	**Version**	**Rôle dans le projet**
Java	17	Langage de programmation principal du backend.
Spring Boot	3.2.0	Framework backend qui gère les routes HTTP, les sessions, la base de données et les WebSockets.
Spring Data JPA	Intégré	Couche d'abstraction pour la communication avec MySQL via les interfaces Repository.
Spring WebSocket	Intégré	Gestion des connexions bidirectionnelles en temps réel pour les hackathons. Protocole STOMP sur SockJS.
Thymeleaf	Intégré	Moteur de templates HTML qui injecte les données du serveur dans les pages web.
MySQL	8.0	Base de données relationnelle pour le stockage des utilisateurs, historiques et hackathons.
Bootstrap	5.3.0	Framework CSS pour la conception de l'interface utilisateur responsive.
JavaScript (ES6)	—	Gestion des interactions utilisateur, des appels APIs et de l'affichage dynamique.
SockJS + STOMP	1.x / 2.3.3	Bibliothèques JavaScript pour la connexion WebSocket côté client.

API Groq (llama-3.3-70b)	—	Modèle d'IA externe pour l'analyse, la correction, l'optimisation et la transformation du code.

Tableau 2.5 — Frameworks et technologies utilisés

2.6 Conclusion :

-Ce deuxième chapitre nous a permis de définir avec précision les besoins fonctionnels et non fonctionnels de Codify, d'identifier les deux acteurs du système et de planifier le travail en 5 sprints répartis sur 3 releases. Nous avons également présenté l'architecture MVC à trois couches adoptée, qui garantit la séparation des responsabilités et la maintenabilité du code, et l'ensemble des technologies utilisées. Les chapitres suivants détailleront la réalisation de chaque release, sprint par sprint, avec les diagrammes UML correspondants, les interfaces réalisées et les explications détaillées du fonctionnement de chaque fonctionnalité.

Chapitre 3

Release 1

3.1 Introduction :

-Le troisième chapitre présente la première Release du projet Codify, Le Sprint 1 est consacré à la gestion des utilisateurs, l'authentification et la sélection du niveau de programmation et du Sprint 2 consacré à l'environnement d'écriture et d'exécution du code. Pour chaque sprint, nous présentons les objectifs, le backlog, l'analyse à travers les diagrammes de cas d'utilisation, la conception à travers les diagrammes de classe et de séquence, ainsi que les interfaces réalisées.

3.2 Sprint 1 : Gestion des utilisateurs et authentification :

-Dans cette partie, nous avons présenté les différentes étapes de réalisation du premier sprint

3.2.1 Objectif du Sprint 1 :

-L'objectif du premier sprint est de développer le module de gestion des utilisateurs. Ce module permet l'utilisateur de créer un compte en choisissant son niveau de programmation parmi trois options : Débutant, Intermédiaire ou Avancé. Il permet également à l'utilisateur de se connecter à son espace personnel et d'accéder à son tableau de bord affichant ses statistiques personnelles (points, niveau, exercices complétés, classement et historique des exercices résolus).

3.2.2 Backlog du Sprint 1 :

ID	Fonctionnalité	User Story	Priorité	Complexité
1.1	Inscription utilisateur	En tant qu'utilisateur, je veux créer un compte pour accéder aux fonctionnalités du site.	Haute	Faible

1.2	Authentification	En tant qu'utilisateur inscrit, je veux me connecter à mon compte pour sauvegarder mon activité.	Haute	Faible
1.3	Choix du niveau	En tant qu'utilisateur inscrit, je veux choisir mon niveau (Débutant, Intermédiaire, Avancé) pour que l'IA adapte ses explications.	Moyenne	Moyenne
1.4	Gestion Admin	En tant qu'administrateur, je veux gérer les comptes utilisateurs.	Moyenne	Moyenne

Table 3.1– Backlog Produit de Sprint 1

3.2.3 Analyse du Sprint 1 :

-Dans cette partie nous allons présenter le diagramme du cas d'utilisation du premier sprint et la description textuelle de chaque cas d'utilisation. La Figure 3.1 présente le diagramme de cas d'utilisation du sprint courant

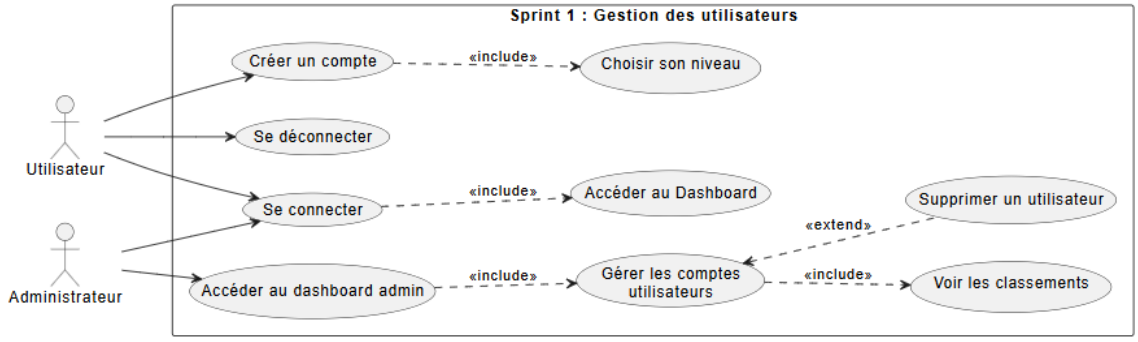


Figure 3.1– Diagramme de cas d'utilisation du Sprint 1

Description des scénarios :

scénarios 1 :Inscription d'un utilisateur:

1. L'utilisateur accède à la page d'accueil et clique sur "S'inscrire"
2. Il saisit son nom d'utilisateur, son adresse email et son mot de passe
3. Il sélectionne son niveau parmi trois options : Débutant, Intermédiaire ou Avancé
4. Il clique sur "Cree mon compte"
5. Si l'email est déjà utilisé, un message d'erreur s'affiche : "Email déjà utilisé !"
6. Sinon, le compte est créé dans la base de données et l'utilisateur est redirigé vers la page de connexion

scénarios 2:Connexion d'un utilisateur:

1. L'utilisateur saisit son email et son mot de passe

2. Il clique sur "Se connecter"
3. Le système vérifie les identifiants dans la base de données
4. Si corrects, l'utilisateur est redirigé vers son tableau de bord personnalisé
5. Sinon si incorrects, un message d'erreur s'affiche : "Email ou mot de passe incorrect !"

Scénarios 3 : Gestion administrateur :

1. L'administrateur saisit son email et son mot de passe
2. Le système vérifie les identifiants dans la base de données
3. Si corrects, L'administrateur est redirigé vers son tableau de bord
4. Il visualise la liste de tous les utilisateurs classés par niveau
5. Il peut supprimer un utilisateur
6. Sinon si incorrects, un message d'erreur s'affiche : "Email ou mot de passe incorrect !"

3.2.4 Conception du Sprint 1 :

a-Diagramme de classe du Sprint 1 :

-Un User est représenté par (id, username, email, password, level, points, exercicesCompletes, tempsTotal, testDisponible). Il possède un niveau qui détermine les exercices qui lui sont proposés.

-Un HistoriqueExercice est caractérisé par (id, userId, titreExercice, niveau, pointsGagnes, tempsSecondes, tempsFormate, complexite, resultat). Il enregistre chaque exercice résolu par un utilisateur avec ses statistiques détaillées et il appartient à un seul utilisateur.

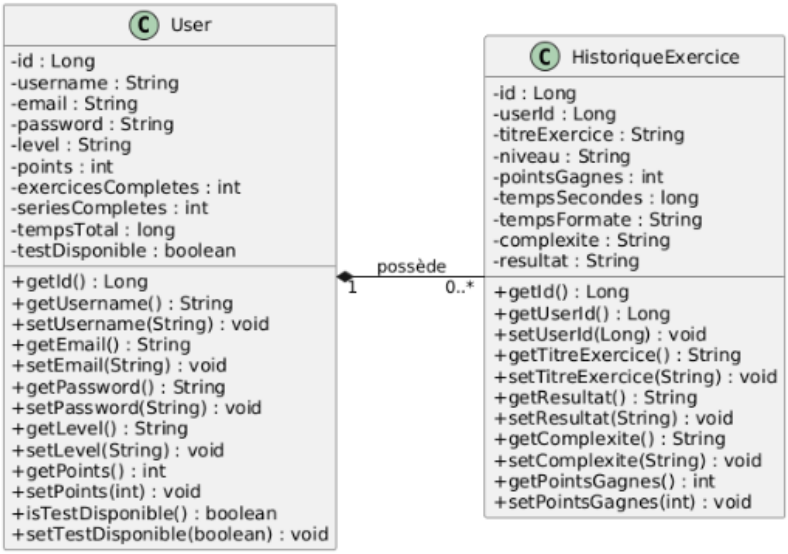


Figure 3.2– Diagramme de classe Sprint 1

-un utilisateur possède zéro ou plusieurs entrées dans son historique. Si l'utilisateur est supprimé, tout son historique est supprimé également.

b- Diagrammes de séquence du Sprint 1 :

- La figure 3.3 montre le diagramme de séquence de notre cas d'utilisation « Inscription» que nous avons élaboré dans ce chapitre

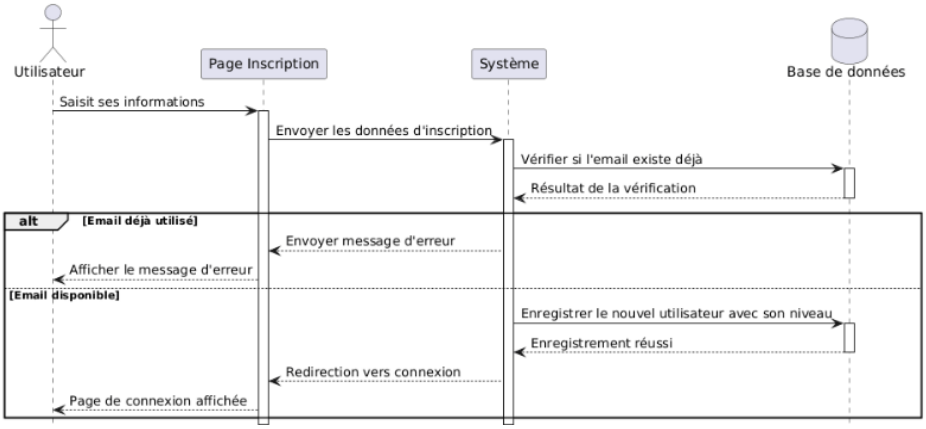


Figure 3.3– Diagramme de séquence : Inscription

Explication de l'exécution :

1. L'utilisateur remplit le formulaire d'inscription
2. Le système vérifie d'abord si l'adresse email exsiste dans la BD ou non
3. Si c'est le cas, un message d'erreur est affiché
4. Sinon, le nouveau compte est créé avec le niveau choisi et les points à zéro
5. L'utilisateur est redirigé vers la connexion.

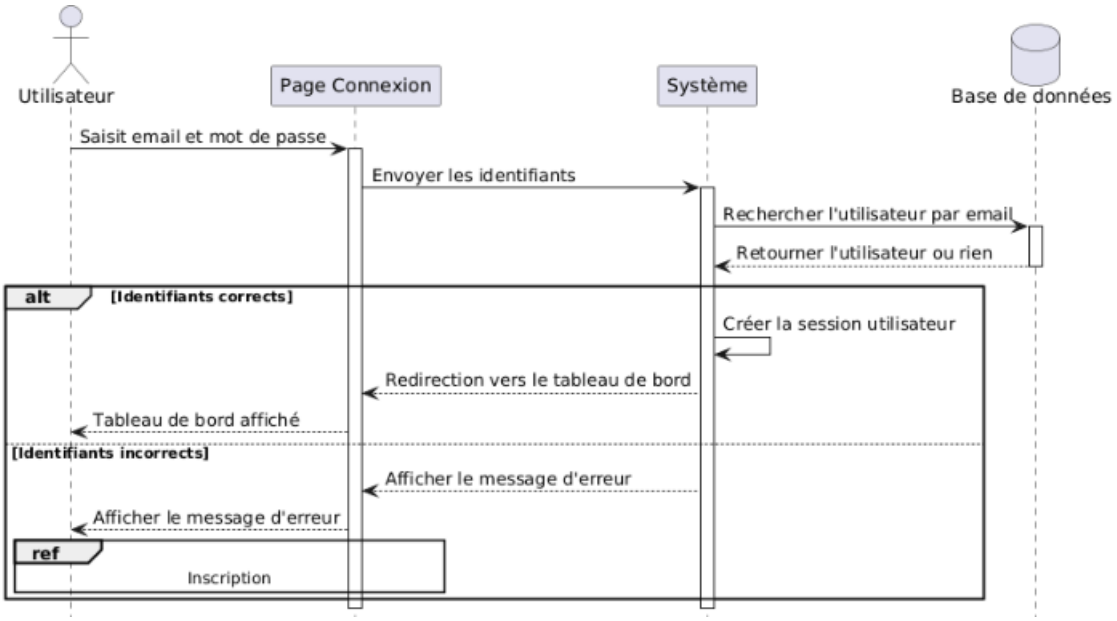


Figure 3.4– Diagramme de séquence : Connexion

-La figure 3.4 montre le diagramme de séquence de notre cas d'utilisation « Connexion » que nous avons élaboré dans ce chapitre

Explication de l'exécution :

1. L'utilisateur saisit ses identifiants.

2. Le système recherche le compte correspondant dans la BD
3. Si trouvé et mot de passe valide, l'objet User est placé en session et l'utilisateur est redirigé vers le Dashboard.
4. Sinon, un message d'erreur est renvoyé à la vue.

3.2.5 Réalisation du Sprint1 :

-Pour mieux comprendre le fonctionnement de notre projet et clarifier le concept, nous avons présenté les interfaces d'application qui fournissent les diverses fonctionnalités spécifiées dans le premier sprint. Ces interfaces ont été préalablement illustrées sous forme de diagrammes afin de vérifier les implications pour la suite du projet

Interface page d'accueil (index.html) :

-La figure 3.5 présente **la page d'accueil de Codify** avec une présentation de statistiques de la plateforme, un aperçu de l'éditeur de code, et les boutons "Créer un compte" et "Se connecter".

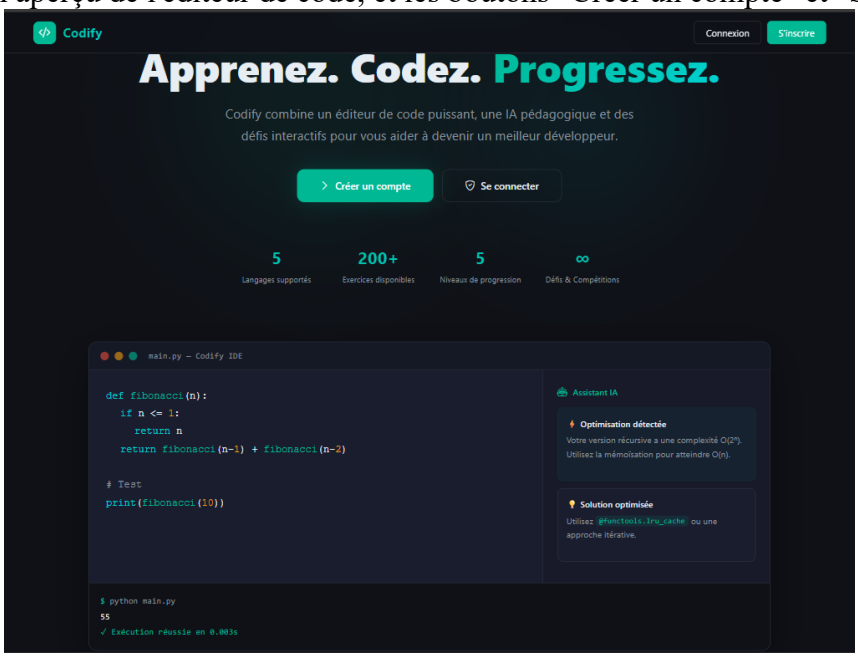


Figure 3.5– page d'accueil

Interface connexion / inscription (login.html) : La figure 3.6 présente l'interface unifiée avec deux onglets : "Connexion" et "Inscription" qui permet aux utilisateurs à accéder à l'application chacun à son propre espace selon niveau.

Et ça sera effectué en passant par le formulaire comme montre la figure ci-dessus qui est composé d'email et mot de passe pour un utilisateur déjà inscrit

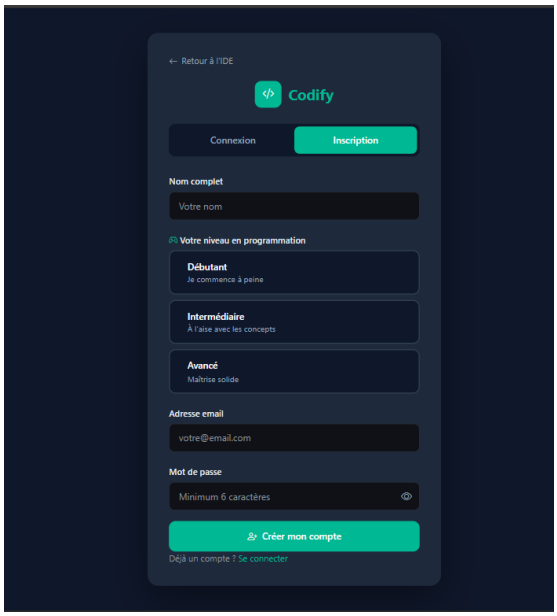


Figure 3.6 l'interface de connexion

-Et la figure 3.7 présente l'interface de l'inscription inclut le nom et la sélection du niveau

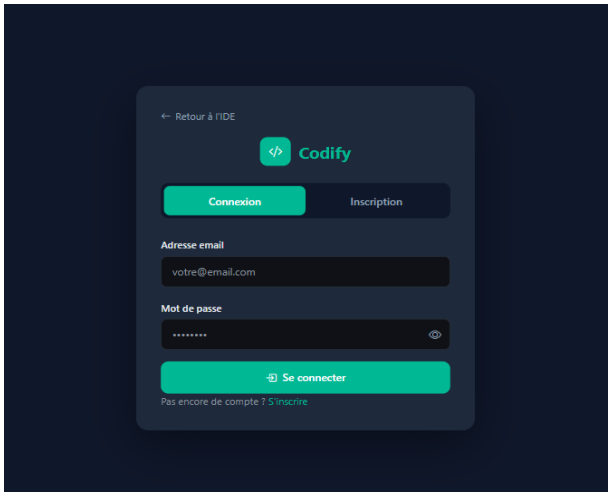


Figure 3.7 l'interface de l'inscription

Interface Dashboard (dashboard.html) : La figure 3.8 présente le tableau de bord avec quatre cartes de statistiques (points, exercices complétés, niveau actuel, classement), le tableau historique des exercices avec colonnes : numéro, exercice, niveau, points, temps, complexité, résultat, et le Top 5 du classement du même niveau

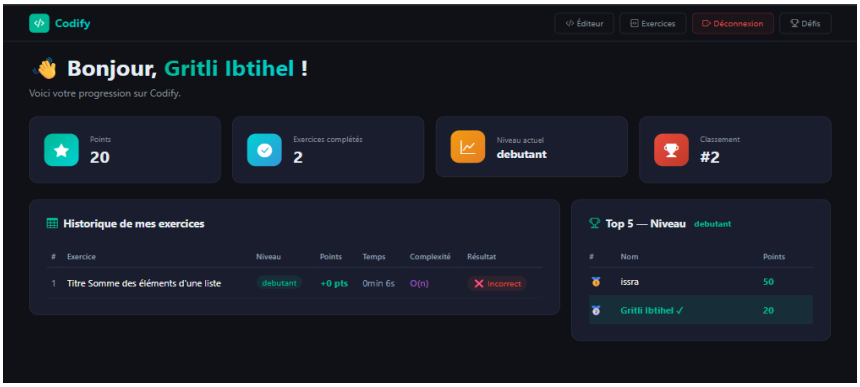


Figure 3.8 tableau de bord de l'utilisateur

Interface admin (admin-login.html) : La figure 3.9 présente l'interface de login de l'admin qui permet à accéder à son Dashboard pour gère les utilisateurs et ça sera effectué en passant À son propre espace par le formulaire comme montre la figure ci-dessus qui est composé d'email et mot de passe

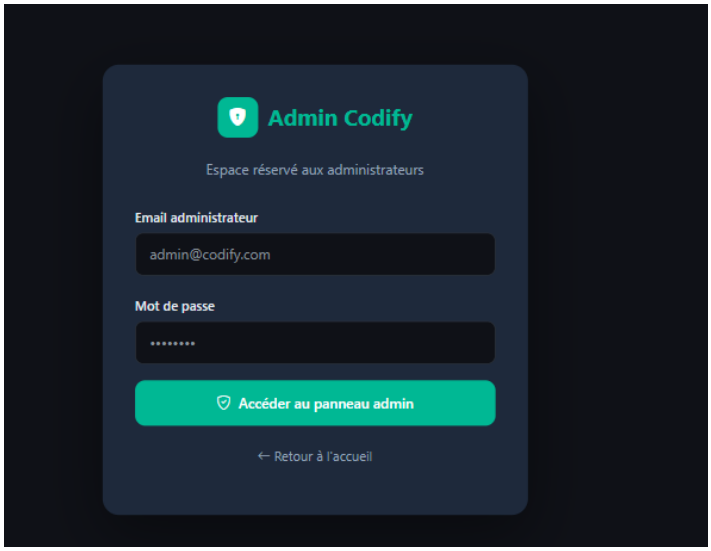


Figure 3.9 l'interface de connexion de l'admin

Interface Dashboard de l'admin (admin-dashboard.html) : La figure 3.10 présente le panneau administrateur avec le bilan, les 3 classements et la liste complète des utilisateurs.

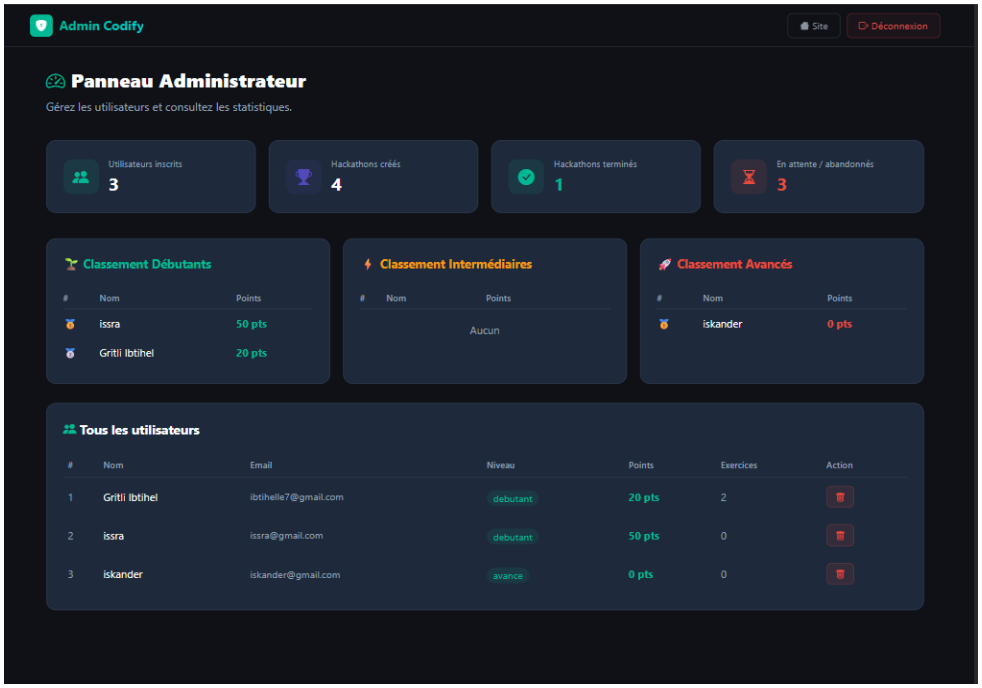


Figure 3.10 le panneau administrateur

3.3 Sprint 2 : Éditeur de code et exécution :

-Dans cette partie, nous allons présenter les étapes de réalisation du deuxième sprint.

3.3.1 Objectif du Sprint 2 :

-L'objectif du deuxième sprint est de développer l'éditeur de code interactif qui permet à l'utilisateur d'écrire du code dans trois langages (Python, Java, C) de l'exécuter et affiche également les erreurs de syntaxe ou de compilation si le code contient des problèmes, ainsi que de transformer son code d'un langage vers un autre.

3.3.2 Backlog du Sprint 2 :

-Le tableau 3.2 représente le back log du sprint 2

ID	Fonctionnalité	User Story	Priorité	Complexité
2.1	Éditeur de code	En tant qu'utilisateur, je veux écrire du code en différents langages.	Haute	Moyenne
2.2	Multi-langage	En tant qu'utilisateur, je veux pouvoir choisir entre plusieurs langages.	Haute	Élevée
2.3	Exécution du code	En tant qu'utilisateur, je veux exécuter mon code et voir le résultat.	Haute	Moyenne
2.4	Affichage des erreurs	En tant qu'utilisateur, je veux voir les erreurs si mon code ne fonctionne pas.	Haute	Moyenne

Table3.2–BacklogProduitduSprint2

3.3.3 Analyse du sprint 2 :

-La figure 3.11 présente le diagramme de cas d'utilisation lié à Environnement d'écriture et d'exécution du code

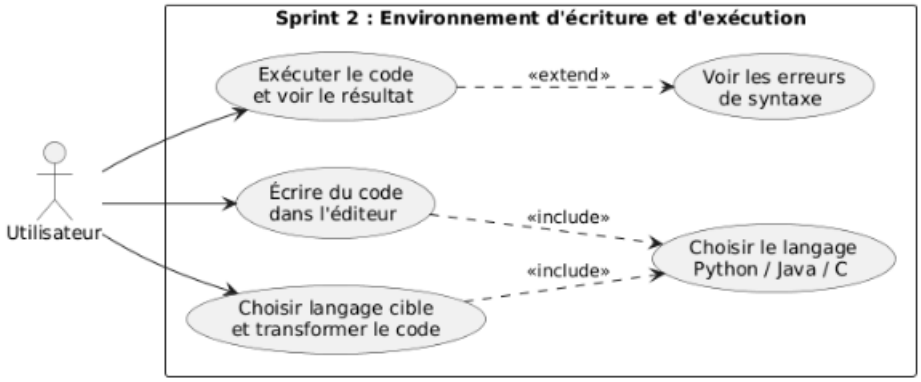


Figure 3.11– Diagramme de cas d'utilisation Sprint 2

Description des scénarios :

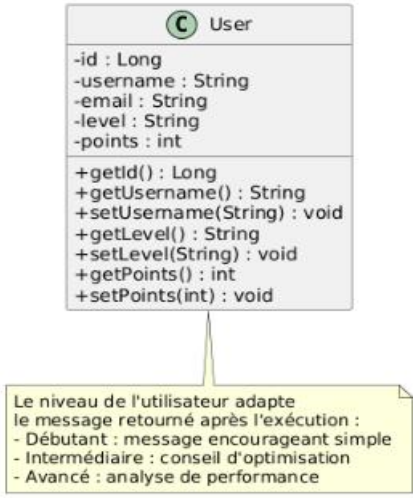
Scénarios1 :

1. L'utilisateur accède à la page /ide.
2. Il sélectionne un langage dans la liste déroulante (Python par défaut).
3. Il écrit son code source dans la zone de texte.
4. Il clique sur le bouton Exécuter.
5. Le code est analysé et le résultat d'exécution est afficher dans le panneau à droite (erreur, syntaxe, …)

3.3.4 Conception du Sprint 2 :

a- Diagramme de classe du Sprint 2 :

- User : la même entité que le Sprint 1. Le champ niveau est utilisé ici pour adapter le message retourné après l'exécution du code.



La figure 3.12 diagramme de classe du sprint 2

b- Diagrammes de séquence du Sprint 2 :

La figure 3.13 montre le diagramme de séquence de notre cas d'utilisation « Exécution du code » que nous avons élaboré dans ce chapitre.

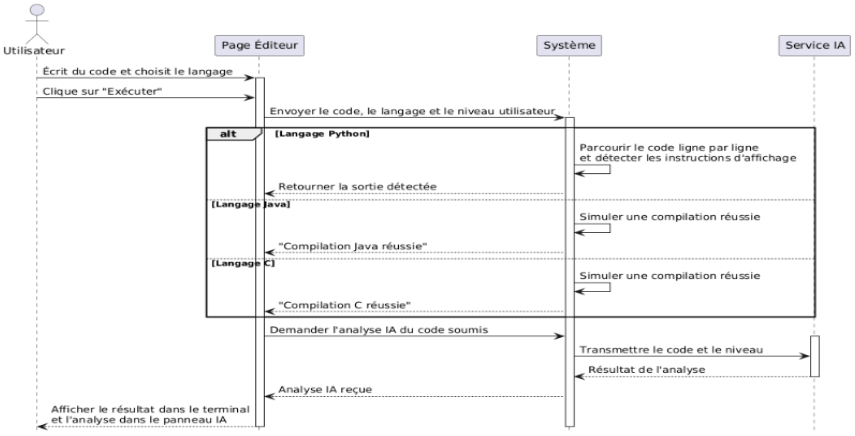


Figure 3.13– Diagramme de séquence : Exécution du code

Explication de l'exécution :

1. L'utilisateur écrit son code et choisit son langage d'écriture
2. Il clique sur exécuter, le système récupère d'abord les informations de l'utilisateur connecté pour connaître son niveau.
3. Ensuite, selon le langage choisi, deux comportements différents se produisent :
4. Si Python en parcourir le code ligne par ligne
5. Sinon si pour Java et C, en simulie rune compilation
6. Après l'exécution, l'analyse IA est appelée automatiquement pour analyser le code et afficher les erreurs détecter

3.3.5 Réalisation du Sprint 2 :

-Pour mieux comprendre le fonctionnement de notre projet et clarifier le concept, nous avons présenté les interfaces d'application qui fournissent les diverses fonctionnalités spécifiées dans le deuxième sprint. Ces interfaces ont été préalablement illustrées sous forme de diagrammes afin de vérifier les implications pour la suite du projet.

Interface éditeur de code(ide.html): La figure 3.14 présente l'interface avec la zone de code (textarea monospace), les sélecteurs de langages source/cible, les boutons "Transformer" et "Exécuter", la zone terminal de résultat, et le panneau IA à droite avec 4 onglets.

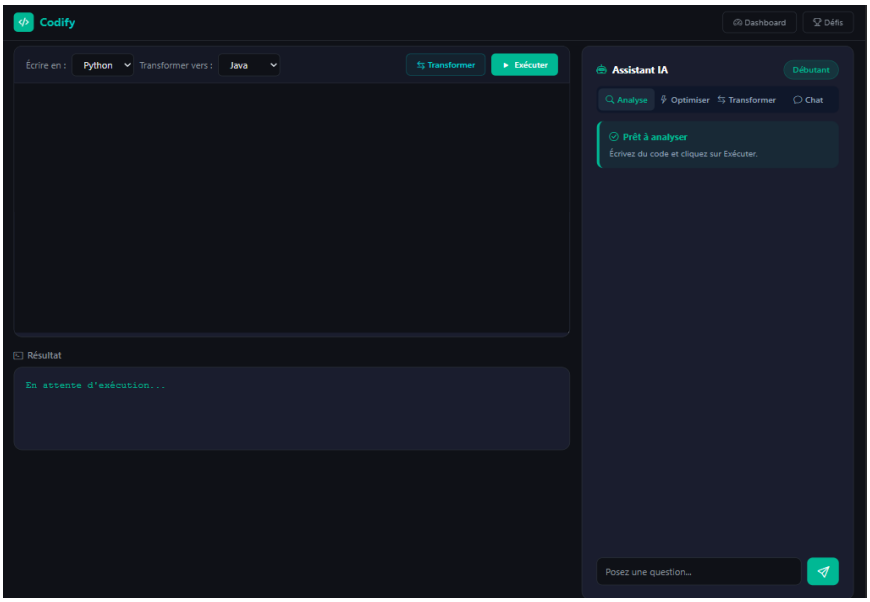


Figure 3.14– Interface de l'ide

Interface résultat d'exécution : La figure 3.15 présente l'affichage du résultat d'exécution dans le panneau IA avec les onglets Analyse, Optimiser, Transformer et Chat.

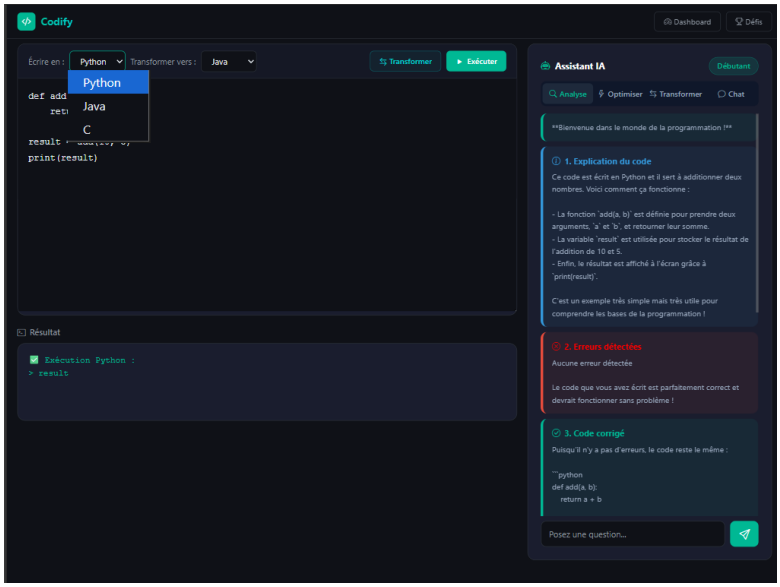


Figure 3.14– Interface résultat d'exécution

3.4 Conclusion :

-Cette première Release a établi les fondations solides du projet Codify. Le Sprint 1 a mis en place un système d'authentification complet avec gestion des niveaux adaptatifs, un tableau de bord dynamique avec historique détaillé et classement par niveau. Le Sprint 2 a introduit l'éditeur de code interactif et la structure du panneau IA qui sera enrichi dans la Release 2.

Chapitre 4

Release 2

4.1 Introduction :

-Le quatrième chapitre présente la deuxième Release de Codify, composée du Sprint 3 consacré à l'analyse intelligente du code par l'IA, et du Sprint 4 dédié à la transformation et la comparaison des langages. Cette Release représente le cœur intelligent de la plateforme grâce à l'API Groq avec des prompts entièrement personnalisés selon le niveau de l'utilisateur.

4.2 Sprint 3 : Analyse intelligente du code (IA) :

-Dans cette partie, nous allons présenter les différentes étapes de réalisation du 3ème sprint

4.2.1 Objectif du Sprint 3 :

-L'objectif du troisième sprint est de développer le système d'analyse intelligente du code par l'IA, avec des prompts adaptés au niveau de l'utilisateur. L'IA détecte les erreurs, propose des corrections, explique le code et suggère des optimisations selon le niveau de l'utilisateur.

4.2.2 Backlog du Sprint 3 :

ID	Fonctionnalité	User Story	Priorité	Complexité
3.1	IA simulée locale	En tant qu'utilisateur, je veux que le système détecte mes erreurs simples et m'indique la ligne exacte du problème.	Haute	Élevée
3.2	Assistant IA externe pédagogique	En tant qu'utilisateur, je veux qu'une IA externe m'explique mes erreurs complexes de manière détaillée et adaptée selon mon niveau.	Haute	Élevée

3.3	Explication des erreurs	En tant qu'utilisateur, je veux comprendre la cause de l'erreur qui rend mon code incorrect.	Haute	Élevée
3.4	Correction proposée	En tant qu'utilisateur, je veux recevoir une proposition de solution corrigée de mon code avec explication des modifications.	Moyenne	Moyenne
3.5	Optimisation du code	En tant qu'utilisateur, je veux que l'IA propose une version plus optimale même si mon code fonctionne, avec explication.	Moyenne	Élevée
3.6	Conseils personnalisés	En tant qu'utilisateur, je veux recevoir des conseils pour améliorer ma logique et mes méthodes de codage.	Moyenne	Moyenne

Table 4.3– Backlog Produit du Sprint 3

4.2.3 Analyse du Sprint 3 :

-Dans cette partie nous allons présenter le diagramme du cas d'utilisation du troisième sprint et la description textuelle de chaque cas d'utilisation. La figure 4.2 présente le diagramme de cas d'utilisation du sprint 2. Ce cas d'utilisation permet de faire l'explication du code, la détection des erreurs avec indication de la ligne,une proposition de code corrigé,une version optimisée (selon le niveau) et des conseils personnalisés adaptés au niveau de l'utilisateur.

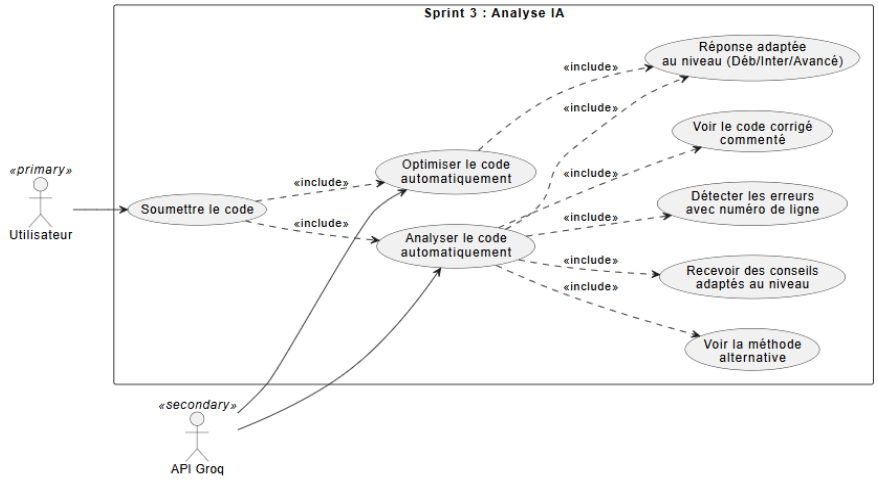


Figure 4.2– Diagramme de cas d'utilisation du sprint 2

Description des scénarios :

Scénario : Analyser le code automatiquement deouis IA

1. L'utilisateur exécute son code dans l'éditeur.

2. Le système déclenche automatiquement analyser avec le code, le langage et le niveau.
3. L'IA analyse le code en profondeur selon le niveau de l'utilisateur.
4. L'API renvoie une réponse textuelle structurée (explication, erreurs, code corrigé, conseils) et affichée dans le panneau IA de l'IDE, dans l'onglet « Analyse ».

Scénario : Optimiser le code

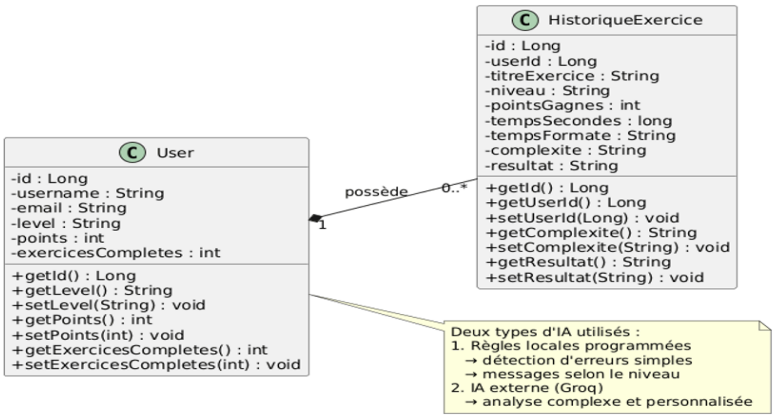
1. Après l'exécution, l'IA propose automatiquement une version améliorée du code.
2. La version optimisée est adaptée au niveau de l'utilisateur.
3. Les explications des améliorations s'affichent dans l'onglet Optimiser.

4.2.4 Conception du Sprint 3 :

a- Diagramme de classe du Sprint 3 :

-User : l'entité utilisateur. Le champ niveau détermine le type d'explications que l'IA retourne (simples pour débutant, techniques pour avancé).

-HistoriqueExercice : enregistre les résultats de chaque exercice corrigé, y compris la complexité algorithmique extraite de la réponse de l'IA.



4.3 Diagramme de classe du sprint 3

b- Diagrammes de séquences du Sprint 3 :

-La figure 4.4 représente le diagramme de séquence de notre cas d'utilisation « Analyse IA adaptée au niveau».

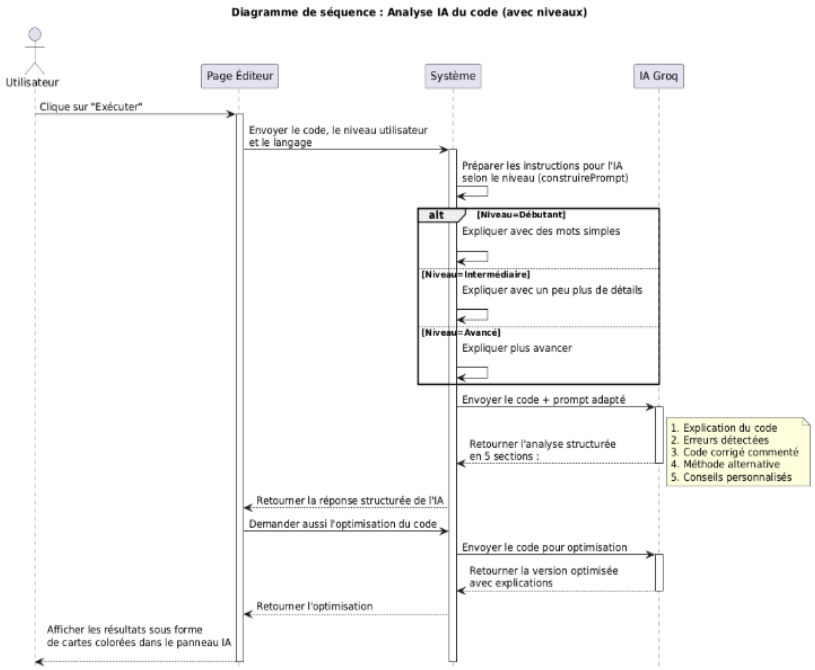


Figure 4.4– Diagramme de séquence d'ajout d' Analyse IA adaptée au niveau

Explication de l'exécution :

1. L'utilisateur clique sur « Exécuter » dans la page éditeur.
2. La page éditeur envoie au système le code source, le niveau de l'utilisateur (Débutant, Intermédiaire ou Avancé) et le langage de programmation.
3. Le système prépare les instructions (construit le « prompt ») en fonction du niveau :
4. Si Débutant → expliquer avec des mots très simples.
5. Si Intermédiaire → expliquer avec un peu plus de détails techniques.
6. Si Avancé → expliquer de manière plus avancée (performances, patterns).
7. Le système envoie le code + le prompt adapté à l'IA (API Groq).
8. L'IA retourne une analyse structurée en 5 sections : Explication du code, Erreurs détectées, Code corrigé commenté, Autre solution et Conseils personnalisés
9. Le système transmet cette réponse structurée à la page éditeur.
10. La page éditeur demande également l'optimisation du code au système.
11. Le système envoie le code à l'IA pour optimisation.
12. L'IA retourne la version optimisée du code, accompagnée d'explications.
13. Le système renvoie cette optimisation à la page éditeur.
14. La page éditeur affiche les résultats sous forme de cartes colorées dans le panneau IA

4.2.5 Réalisation du Sprint 3 :

-Pour mieux comprendre le fonctionnement de notre projet et clarifier le concept, nous avons présenté les interfaces d'application qui fournissent les différentes fonctionnalités spécifiées dans le troisième sprint. Ces interfaces ont été préalablement illustrées sous forme de diagrammes afin de vérifier les implications pour la suite du projet.

Interface panneau IA — Analyse : La figure 4.5 présente le panneau IA avec l'onglet Analyse affichant les résultats sous forme de cartes colorées : rouge pour les erreurs détectées, vert pour le code corrigé, bleu pour les explications, jaune pour les conseils

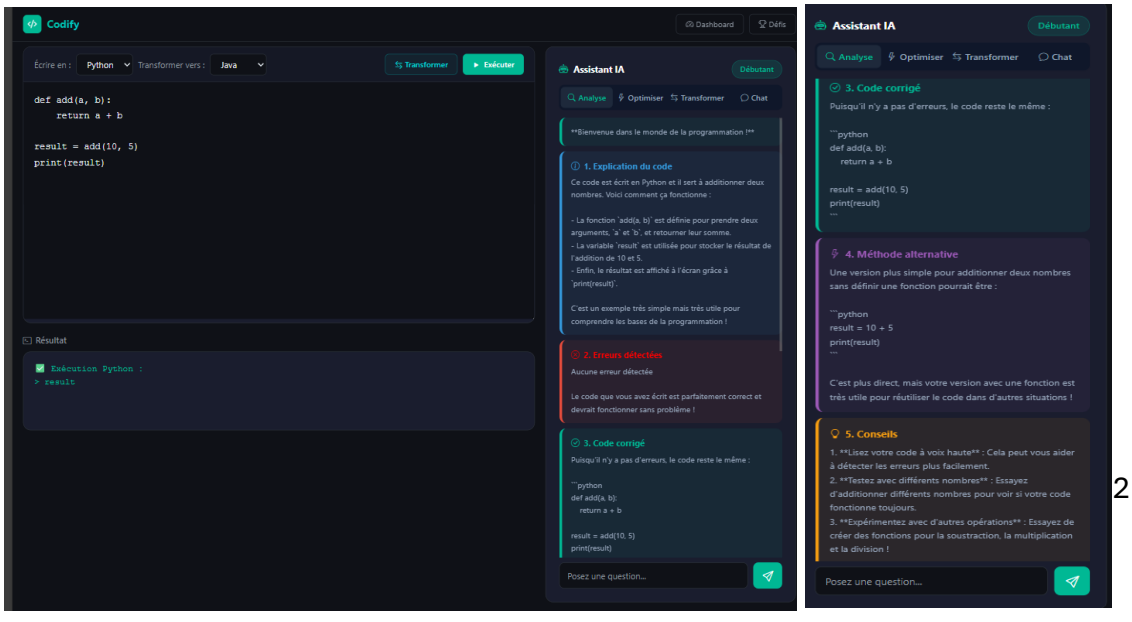


Figure 4.5– Interface panneau IA — Analyse

Interface panneau IA — Optimisation : La figure 4.6 présente l'onglet Optimiser avec la version optimisée du code et les explications des améliorations apportées selon le niveau de l'utilisateur.

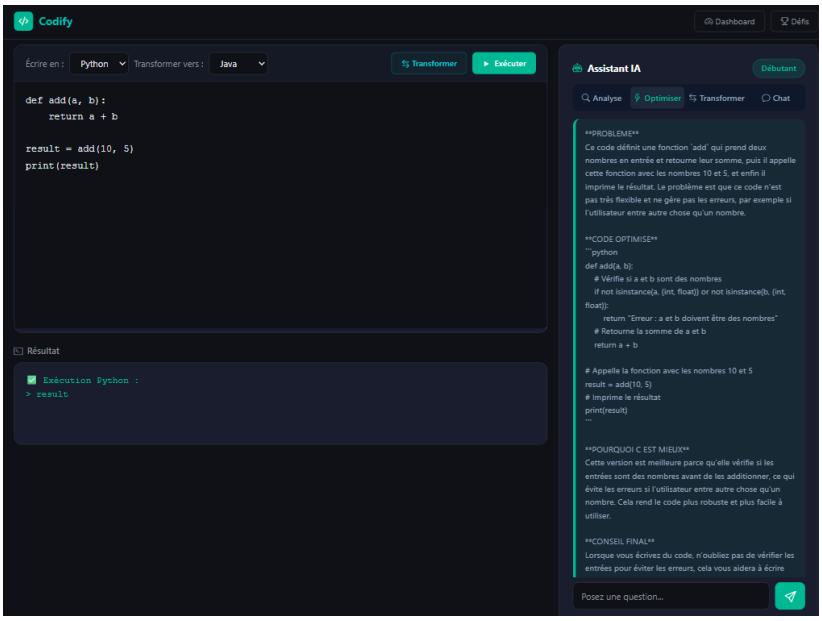


Figure 4.6– Interface panneau IA — Optimisation

4.3 Sprint 4 : Transformation de code :

-Dans cette partie, nous allons présenter les différentes étapes de réalisation du 4ème sprint

4.3.1 Objectif du Sprint 4 :

-L'objectif du quatrième sprint est de permettre à l'utilisateur de transformer son code d'un langage de programmation vers un autre (Python → Java, Java → C, etc.) avec des explications des différences de syntaxe, de logique et de performance et analyse quel langage est le plus adapté pour résoudre un problème.

4.3.2 Backlog du Sprint 4 :

ID	Fonctionnalité	User Story	Priorité	Complexité
4.1	Transformation de code	En tant qu'utilisateur, je veux transformer mon code d'un langage vers un autre.	Haute	Élevée
4.2	Explication de la syntaxe	En tant qu'utilisateur, je veux comprendre les différences entre les langages.	Haute	Moyenne

4.3	Comparaison des langages	En tant qu'utilisateur, je veux savoir quel langage est le plus adapté pour résoudre mon problème.	Moyenne	Élevée

Table 4.4– Backlog Produit du Sprint 4

4.3.3 Analyse du Sprint 4 :

-Dans cette partie, nous allons présenter le diagramme du cas d'utilisation du quatrième sprint et la description textuelle de chaque cas d'utilisation.

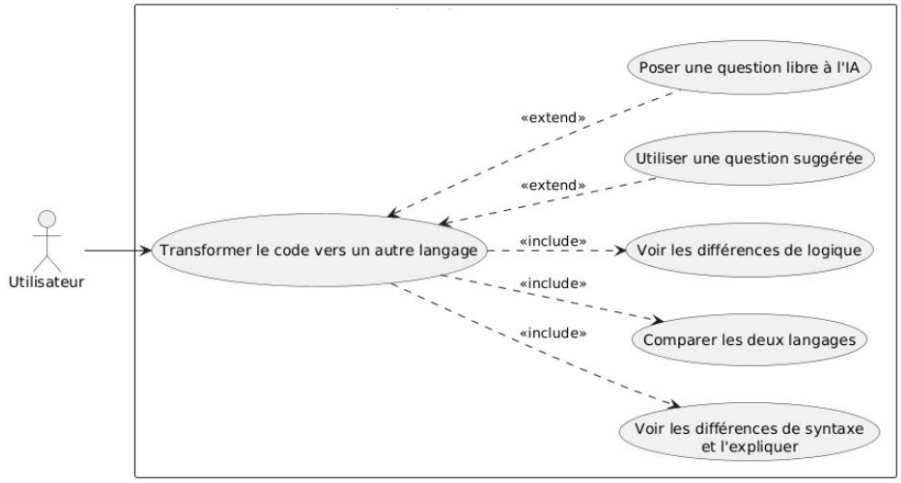


Figure 4.9– Diagramme de cas d'utilisation du Sprint 4

Description des scénarios

Scénario : Transformer le code

1. L'utilisateur choisit le langage dans lequel son code est écrit.
2. Il choisit le langage vers lequel il veut transformer son code.
3. Il clique sur le bouton de transformation.
4. Si les deux langages sont identiques, un avertissement s'affiche et l'opération est annulée.
5. Sinon, l'IA transforme le code en gardant la même logique.
6. La réponse contient le code transformé, les différences de syntaxe, les différences de logique, une comparaison des deux langages et des conseils.
7. Le code transformé s'affiche dans le terminal et les explications dans le panneau IA.

4.3.4 Conception du Sprint 4 :

a- Diagramme de classe du Sprint 4

-User : la même entité. Le niveau adapte ici le ton et la complexité des explications de transformation



Figure 4.9– Diagramme de cas d'utilisation du Sprint 4

b- Diagrammes de séquence du Sprint 4

-la figure represente le diagramme du sequence de notre cas d'utilisation «Transformation de code»

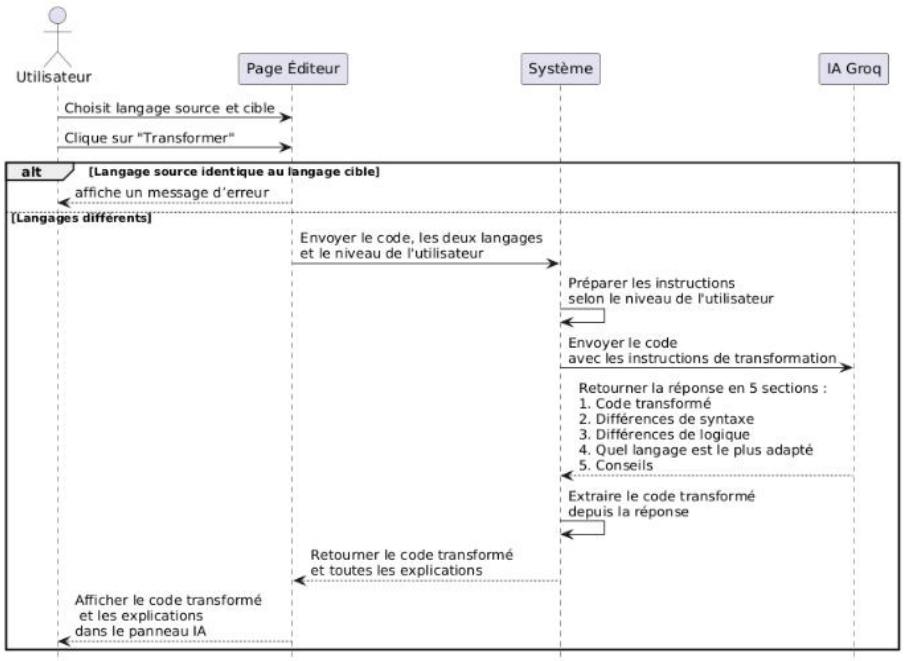


Figure 4.11– Diagramme de sequence : Transformation de code

Explication de l'exécution :

1. L'utilisateur choisit le langage source et le langage cible dans l'interface.
2. L'utilisateur clique sur le bouton Transformer .
3. La page Éditeur vérifie si les deux langages sont identiques :
4. Si langage source = langage cible alors message d'erreur est affiche
5. Sinon poursuit l'exécution.
6. La page Éditeur envoie au Système le code source, les deux langages ainsi que le niveau de l'utilisateur .

7. Le Système construit le prompt pour l'IA en fonction du niveau de l'utilisateur
8. Le Système envoie le code source avec les instructions de transformation à l'IA Groq.
9. L'IA Groq analyse la demande et retourne une réponse structurée en 5 sections :Code transformé ,Différences de syntaxe ,Différences de logique,Quel langage est le plus adapté et Conseils
10. Le Système extrait le code transformé depuis la réponse
11. Le Système transmet les informations à la page Éditeur
12. La page Éditeur affiche Le code transformé et les explications dans le panneau IA

4.3.5 Réalisation du Sprint 4 :

-Pour mieux comprendre le fonctionnement de notre projet et clarifier le concept, nous avons présenté les interfaces de l'application fournissant les diverses fonctionnalités spécifiées dans le quatrième sprint. Ces interfaces ont été préalablement illustrées sous forme de diagrammes afin de vérifier les implications pour la suite du projet

Interface transformation de code : La figure 4.9 présente l'interface de transformation avec les sélecteurs de langages source et cible, le résultat du code transformé dans la zone terminal, et les explications détaillées dans l'onglet Transformer du panneau IA.

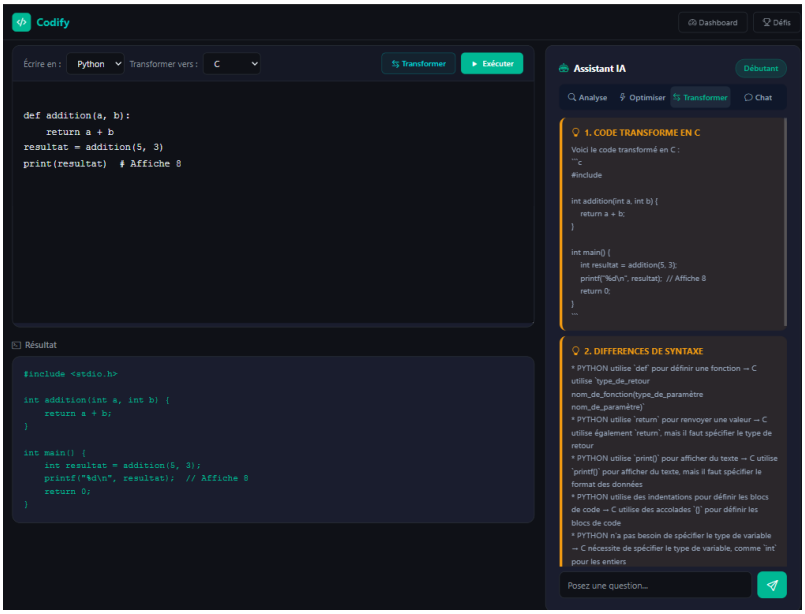


Figure 4.13– Interface de l'interface de transformation de code

Interface chat IA : La figure 4.10 présente l'interface du chat IA interactif avec les questions suggérées, le fil de conversation entre l'utilisateur et l'IA, et le champ de saisie pour poser de nouvelles questions.

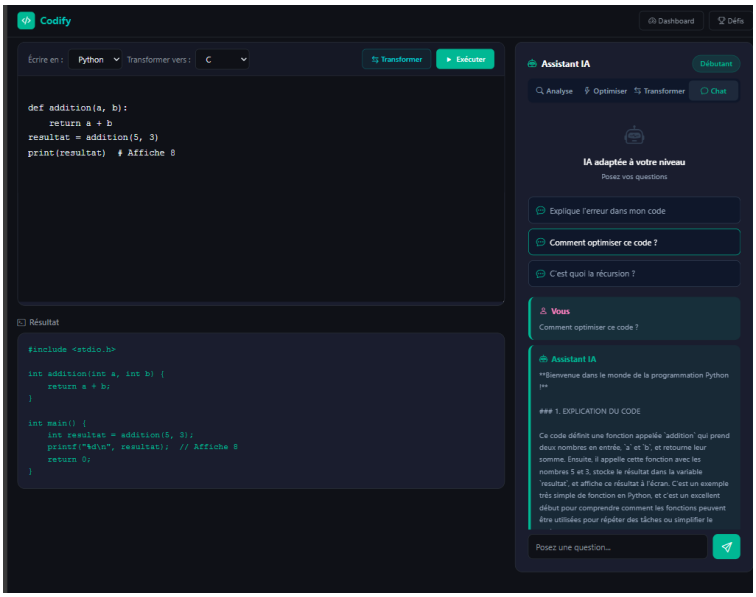


Figure 4.14– Interface chat IA

4.4 Conclusion :

-La deuxième Release a considérablement enrichi les capacités intelligentes de Codify. Le Sprint 3 a mis en place un système d'analyse IA profondément adaptatif avec trois niveaux de prompts distincts, tandis que le Sprint 4 a ajouté la transformation multi-langages et un chat contextuel. L'ensemble de ces fonctionnalités positionne Codify comme un véritable assistant pédagogique IA.

Chapitre 5

Release 3

5.1 Introduction :

-Le cinquième et dernier chapitre présente la troisième Release du projet Codify, la plus complète, composée du Sprint 5. Ce sprint regroupe le système d'exercices adaptatifs générés par l'IA, la gamification avec un classement par niveau et les hackathons en temps réel via WebSocket. Cette release représente l'aboutissement du projet et la mise en place de toutes les fonctionnalités de gamification et de compétition.

5.2 Sprint 5 : Exercices, Hackathons et Gamification :

-Après avoir terminé les 4 premiers sprints, nous avons passé au dernier sprint

5.2.1 Objectif du Sprint 5 :

-L'objectif du cinquième sprint est de développer l'ensemble du système gamifié de Codify : génération d'exercices par l'IA selon le niveau, correction intelligente avec attribution de points, test de validation pour changer de niveau et hackathons compétitifs en temps réel entre deux utilisateurs

5.2.2 Backlog du Sprint 5 :

ID	Fonctionnalité	User Story	Priorité	Complexité
5.1	Exercices de programmation	En tant qu'utilisateur inscrit, je veux résoudre des problèmes de code adaptés à mon niveau pour améliorer mes compétences.	Haute	Moyenne
5.2	Test de niveau	En tant qu'utilisateur, je veux passer un test après une série d'exercices terminée pour vérifier ma progression.	Haute	Moyenne

5.3	Mise à jour du niveau	En tant qu'utilisateur, je veux que mon niveau soit mis à jour selon le résultat du test.	Haute	Faible
5.4	Dashboard	En tant qu'utilisateur, je veux voir mes exercices, le temps, la complexité et le classement des utilisateurs selon les points.	Moyenne	Moyenne
5.5	Défis	En tant qu'utilisateur, je veux participer à des défis en équipe.	Moyenne	Faible
5.6	Récompenses	En tant qu'utilisateur, je veux recevoir une récompense ou une formation si je suis parmi les meilleurs.	Moyenne	Faible

Tableau 5.5 — Backlog Produit du Sprint 5

5.2.3 Analyse du Sprint 5 :

-Dans cette partie, nous allons présenter le diagramme du cas d'utilisation du cinqième sprint et la description textuelle de chaque cas d'utilisation.

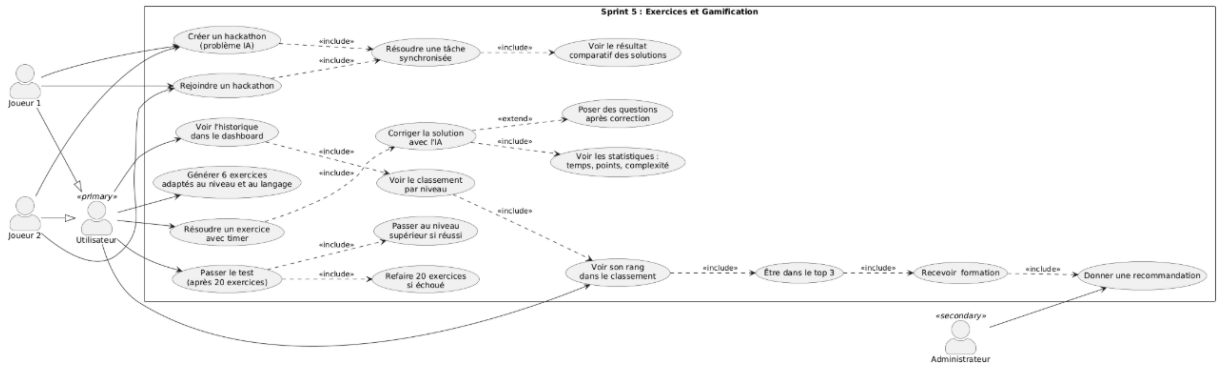


Figure 5.2– Diagramme de cas d'utilisation du Sprint 5

Description des scénarios :

Scénario : Résoudre un exercice

1. L'utilisateur choisit le langage de programmation.
2. Il clique sur "Nouveaux exercices" pour générer une nouvelle série.
3. L'IA génère 6 exercices différents adaptés à son niveau.
4. L'utilisateur clique sur "Commencer" pour ouvrir un exercice.
5. Un timer démarre pour mesurer le temps de résolution.
6. L'utilisateur écrit sa solution et clique sur "Corriger".
7. L'IA évalue la solution

8. L'exercice est enregistré dans l'historique du tableau de bord.
9. L'utilisateur peut poser des questions sur l'exercice après correction.

Scénario : Passer le test de niveau

1. Après 20 exercices complétés, le bouton de test se déverrouille.
2. L'utilisateur clique sur "Passer le Test de Niveau".
3. L'IA génère un test selon le niveau de l'utilisateur.
4. Un compte à rebours démarre (30 minutes pour débutant, 20 pour intermédiaire, 15 pour avancé).
5. L'utilisateur répond aux questions avant la fin du temps.
6. Si le temps s'écoule, le test est soumis automatiquement.
7. L'IA évalue les réponses : si 4 ou 5 bonnes réponses, le niveau est amélioré et le compteur est remis à 0.
8. Sinon, l'utilisateur doit refaire 20 exercices pour pouvoir repasser le test.

Scénario : Participer à un hackathon

1. L'utilisateur crée un hackathon en choisissant le langage et la durée.
2. L'IA génère un vrai problème algorithmique.
3. Le hackathon est affiché dans la liste des hackathons disponibles.
4. Un autre utilisateur voit le hackathon et clique sur "Rejoindre".
5. Les deux timers démarrent exactement au même moment grâce à la même date de début.
6. Chaque joueur voit le score de l'adversaire en temps réel.
7. Si un joueur soumet une solution correcte, le hackathon s'arrête immédiatement pour les deux.
8. Si le temps s'écoule, les scores sont comparés et le gagnant est désigné.
9. L'IA compare les deux solutions selon la complexité et l'optimisation.
10. Le gagnant reçoit 50 points supplémentaires.

4.3.4 Conception du Sprint 5:

a- Diagramme de classe du Sprint 5:

-La figure 4.2 représente le diagramme de classe du sprint 3, qui concerne la gestion des enseignants

-User : représente un utilisateur avec son niveau, ses points et son avancement. Il peut participer à plusieurs hackathons et possède un historique de tous ses exercices résolus.

-HistoriqueExercice : enregistre chaque exercice résolu avec ses statistiques complètes. Chaque entrée appartient obligatoirement à un utilisateur.

-Defi : représente un hackathon entre deux utilisateurs. Il contient le problème généré par l'IA, les scores et les temps des deux joueurs ainsi que le gagnant.

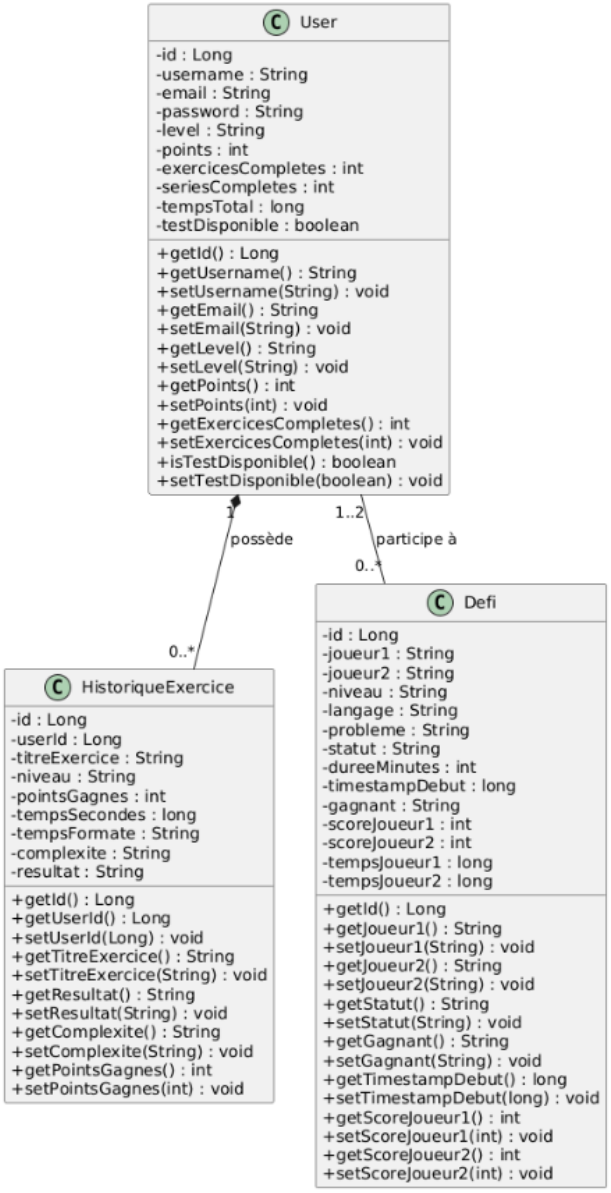


Figure 5.3– Diagramme de classe du Sprint 5

-Composition (1 — 0..*) entre User et HistoriqueExercice : un utilisateur possède zéro ou plusieurs entrées dans son historique. Si l'utilisateur est supprimé, tout son historique disparaît également.

-Association (1..2 — 0..*) entre User et Defi : un hackathon implique exactement 1 ou 2 utilisateurs (le créateur seul en attente, ou les 2 joueurs). Un utilisateur peut participer à zéro ou plusieurs hackathons.

b- Diagrammes de séquence du Sprint 5 :

-la figure represente le diagramme du sequence de notre cas d'utilisation «Résolution d'un exercice»

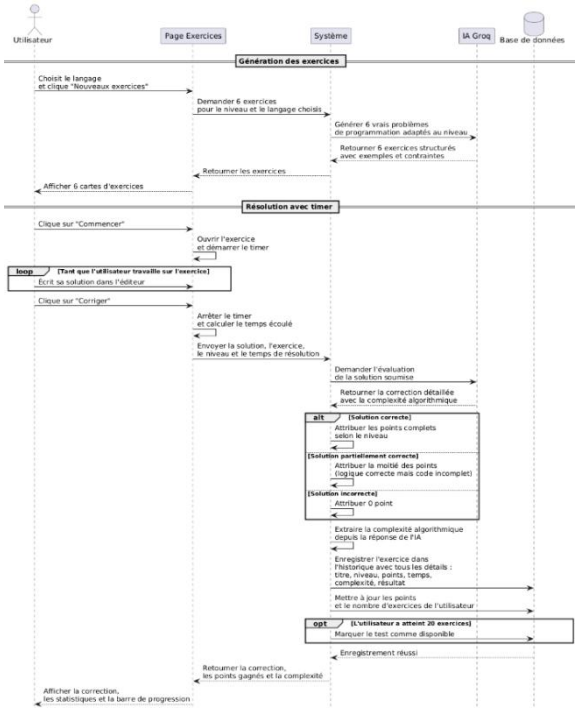


Figure 5.4– Diagramme de sequence : Résolution d'un exercice

Explication de l'exécution :

1. L'utilisateur choisit le langage de programmation et son niveau dans l'interface.
2. L'utilisateur clique sur le bouton Nouveaux exercices.
3. La page Exercices envoie au Système une demande de 6 exercices adaptés au niveau et au langage choisis.
4. Le Système construit un prompt pour l'IA en incluant le niveau et le langage.
5. Le Système envoie la demande de génération d'exercices à l'IA Groq.
6. L'IA Groq génère 6 problèmes de programmation réels, structurés avec énoncés, exemples et contraintes.
7. L'IA Groq retourne les 6 exercices au Système.

8. Le Système transmet les exercices à la page Exercices qui les affiche sous forme de cartes.
9. L'utilisateur choisit un exercice et clique sur Commencer.
10. La page Exercices ouvre l'éditeur de code et démarre un chronomètre local.
11. L'utilisateur écrit sa solution dans l'éditeur
12. L'utilisateur clique sur le bouton Corriger.
13. La page Exercices arrête le chronomètre et calcule le temps écoulé.
14. La page Exercices envoie au Système la solution, l'énoncé de l'exercice, le niveau de l'utilisateur et le temps de résolution.
15. Le Système envoie une demande d'évaluation à l'IA Groq avec le code soumis.
16. L'IA Groq analyse la solution et retourne une correction détaillée
17. Le Système évalue la correction :
18. Si solution correcte → attribution des points complets.
19. Si résolu en moins de 20% du temps maximum du niveau → multiplication des points par 1,5.
20. Si solution partiellement correcte → attribution de la moitié des points.
21. Si solution incorrecte → attribution de 0 point.
22. Le Système extrait la complexité algorithmique depuis la réponse de l'IA.
23. Le Système enregistre dans la Base de données l'historique complet : titre de l'exercice, niveau, points gagnés, temps réalisé, complexité, résultat
24. Le Système met à jour le total de points et le compteur d'exercices de l'utilisateur.
25. Le Système vérifie si le compteur d'exercices atteint 20 :
26. Si oui → le test de niveau devient disponible.
27. Le Système retourne à la page Exercices la correction détaillée, les points gagnés et la complexité algorithmique.
28. La page Exercices affiche les résultats, les statistiques et met à jour la barre de progression.

-la figure 5.4represente le diagramme du sequence de notre cas d'utilisation «Test de validation»

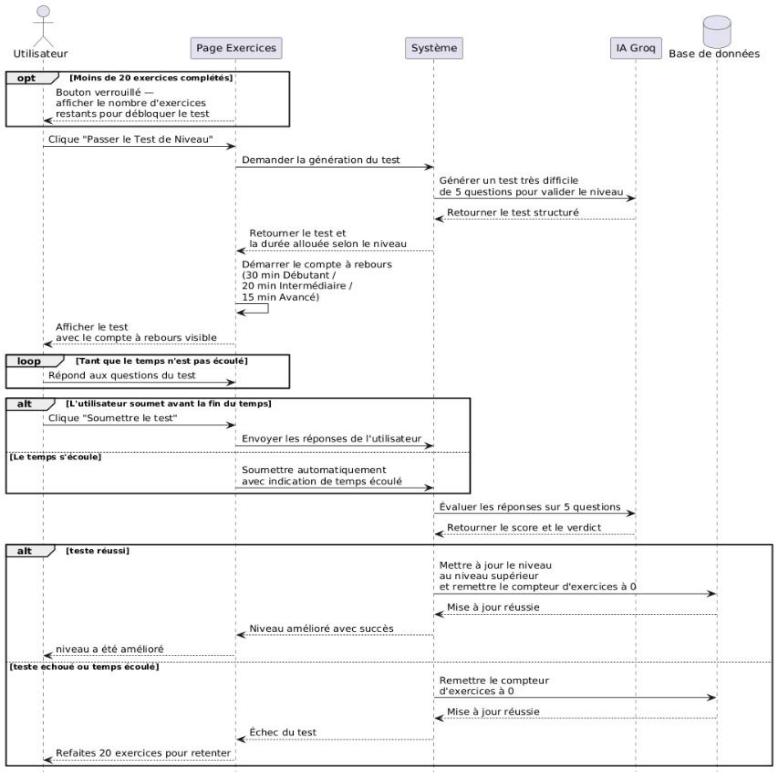


Figure 5.4– Diagramme de sequence : Résolution d'un exercice

Explication de l'exécution :

1. L'utilisateur accède à la page Exercices.
2. Le Système vérifie si l'utilisateur a complété au moins 20 exercices :
3. Si moins de 20 → le bouton Passer le Test de Niveau reste verrouillé avec un message indiquant le nombre d'exercices restants.
4. Si 20 ou plus → le bouton est déverrouillé.
5. L'utilisateur clique sur le bouton Passer le Test de Niveau.
6. La page Exercices envoie une demande de génération du test au Système.
7. Le Système construit un prompt pour l'IA Groq
8. L'IA Groq retourne un test structuré avec 5 questions selon niveau de l'utilisateur
9. Le Système transmet le test à la page Exercices avec la durée allouée selon le niveau :
10. Débutant : 30 minutes
11. Intermédiaire : 20 minutes
12. Avancé : 15 minutes
13. La page Exercices démarre le compte à rebours et affiche le test avec les questions.

14. L'utilisateur répond aux questions dans la limite du temps imparti
15. Cas 1 : L'utilisateur soumet ses réponses avant la fin du temps →La page Exercices envoie les réponses au Système.
16. Cas 2 : Le temps s'écoule →La page Exercices soumet automatiquement les réponses partielles avec une indication "temps écoulé".
17. Le Système envoie les réponses à l'IA Groq pour évaluation.
18. L'IA Groq corrige les 5 questions et retourne le score et le verdict.
19. Le Système analyse le résultat :
20. Si 4 ou 5 bonnes réponses sur 5 → test réussi alors :
21. Le Système met à jour le niveau de l'utilisateur au niveau supérieur.
22. Le compteur d'exercices est remis à 0.
23. Le Système renvoie un message de félicitations.
24. Si moins de 4 bonnes réponses ou temps écoulé → test échoué.
25. Le compteur d'exercices est remis à 0 et l'utilisateur doit refaire 20 exercices
26. Le Système renvoie un message d'encouragement.
27. La page Exercices affiche le résultat final avec un message adapté.

-la figure represente le diagramme du sequence de notre cas d'utilisation «Hackathon»

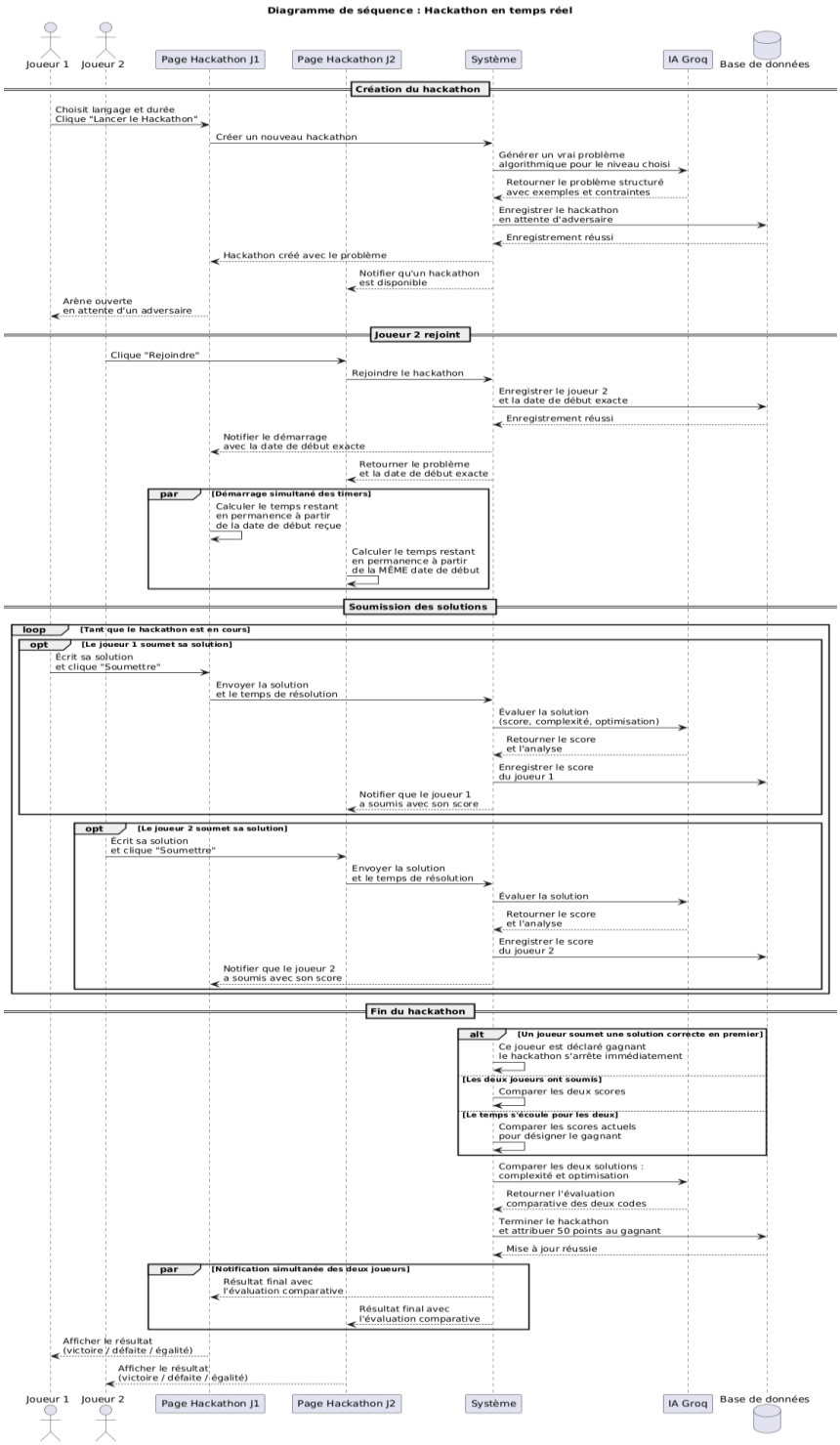


Figure 5.5– Diagramme de sequence : Hackathon

Explication de l'exécution :

1. Création du hackathon par le Joueur 1
2. Le Joueur 1 choisit le langage et la durée du défi, puis clique sur Lancer le Hackathon.
3. La page Hackathon du Joueur 1 envoie une requête de création au Système.
4. Le Système appelle l'IA Groq pour générer un problème algorithmique adapté au niveau du Joueur 1.
5. L'IA Groq retourne le problème structuré avec énoncé, exemples et contraintes.
6. Le Système enregistre le hackathon en base de données avec le statut "en attente d'adversaire".
7. Le Système notifie la page Hackathon du Joueur 2 qu'un hackathon est disponible.
8. La page du Joueur 1 affiche l'arène ouverte avec le problème et un message d'attente.
9. Rejoindre le hackathon par le Joueur 2
10. Le Joueur 2 clique sur Rejoindre dans son interface.
11. La page Hackathon du Joueur 2 envoie une demande d'adhésion au Système.
12. Le Système enregistre le Joueur 2 dans le hackathon et définit une date de début exacte commune.
13. Le Système envoie cette même date de début aux deux pages
14. Démarrage synchronisé des timers
15. Chaque page (J1 et J2) calcule en continu le temps restant à partir de la date de début reçue du serveur.
16. Les deux timers sont ainsi parfaitement synchronisés, même si les navigateurs ne sont pas lancés exactement au même moment.
17. Phase de résolution en parallèle
18. Si le Joueur 1 soumet sa solution :
19. La page du Joueur 1 envoie la solution et le temps écoulé au Système.
20. Le Système verrouille le hackathon (statut "En cours d'évaluation J1") pour éviter les conflits.
21. Le Système envoie la solution à l'IA Groq pour évaluation (score, complexité, optimisation).
22. L'IA Groq retourne le score et l'analyse.
23. Si la solution est correcte et que le hackathon est toujours actif, le Système déclare le Joueur 1 vainqueur immédiat.
24. Si la solution est incorrecte, le Système déverrouille le hackathon et notifie le Joueur 2 que le Joueur 1 a soumis une solution incorrecte.

25. Si le Joueur 2 soumet sa solution :
26. Le même processus se produit symétriquement avec verrouillage et évaluation.
27. Notifications en temps réel :
28. À chaque soumission, le Système envoie une notification à l'adversaire avec le score obtenu.
29. Fin du hackathon
30. Cas A : Un joueur soumet une solution correcte en premier → il est déclaré vainqueur et le hackathon se termine immédiatement.
31. Cas B : Les deux joueurs ont soumis des solutions (correctes ou non) → le Système compare les scores.
32. Cas C : Le temps imparti s'écoule → le Système compare les scores actuels pour désigner le vainqueur.
33. Le Système envoie les deux solutions à l'IA Groq pour une évaluation comparative (analyse de la complexité, optimisation, lisibilité).
34. L'IA Groq retourne un verdict comparatif.
35. Le Système enregistre le résultat final en base de données et attribue 50 points au vainqueur.
36. Le Système envoie simultanément le résultat final et l'évaluation comparative aux pages des deux joueurs.
37. Chaque page affiche le résultat : Victoire, Défaite ou Égalité

4.3.5 Réalisation du Sprint 5 :

-Pour mieux comprendre le fonctionnement de notre projet et clarifier le concept, nous avons présenté les interfaces de l'application fournissant les diverses fonctionnalités spécifiées dans le cinquième sprint. Ces interfaces ont été préalablement illustrées sous forme de diagrammes afin de vérifier les implications pour la suite du projet

Interface exercices adaptés par niveau : La figure 5.6 présente la page des exercices avec 6 cartes générées dynamiquement par l'IA, la barre de progression vers le test de niveau (X/20 exercices), et les boutons de sélection du langage.

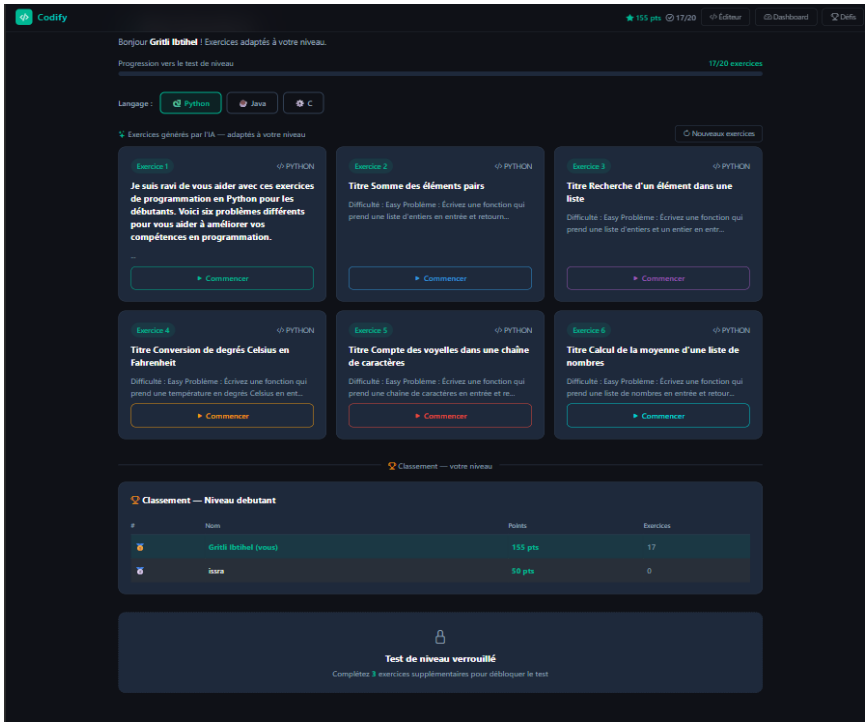


Figure 5.6– Interface de la page des exercices

Interface modal exercice avec timer : La figure 5.7 présente le modal d'exercice avec l'énoncé du problème, le timer en cours, l'éditeur de code, et la zone de statistiques affichant temps, points gagnés et complexité algorithmique après correction.

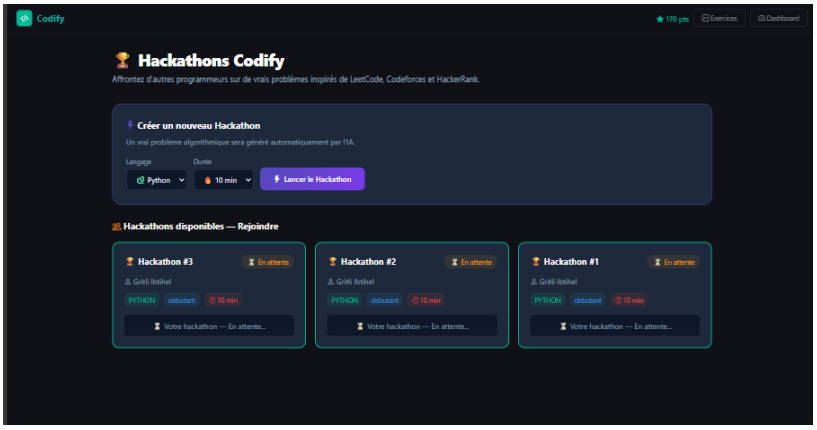


Figure 5.7– Interface de modal d'exercice

Interface hackathon — arène : La figure 5.8 présente un modal hackathon avec une liste de langage de hackaton, une liste pour le temps et les hackathon disponible et la figure 5.9présente l'arène du hackathon avec le compte à rebours synchronisé, les scores des deux joueurs en temps réel, le problème algorithmique, et l'éditeur de code pour soumettre la solution.

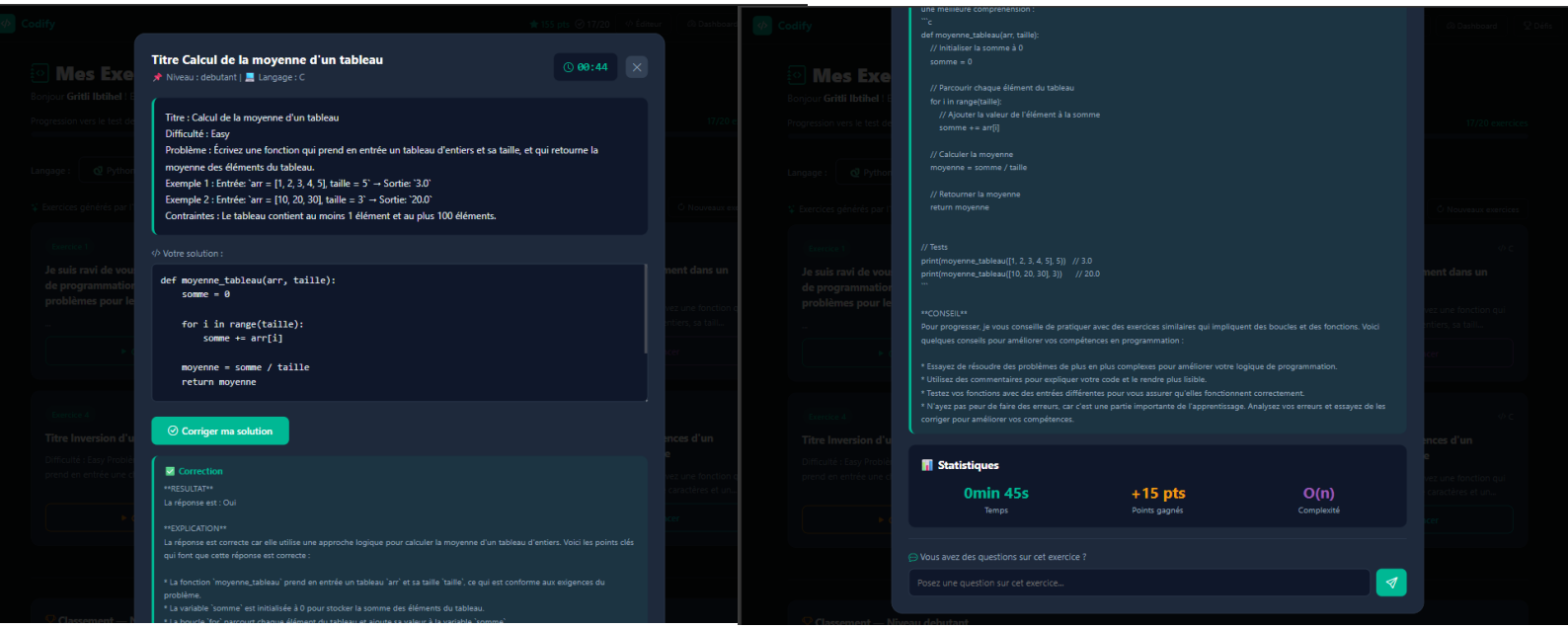


Figure 5.8– Interface de modal hackathon

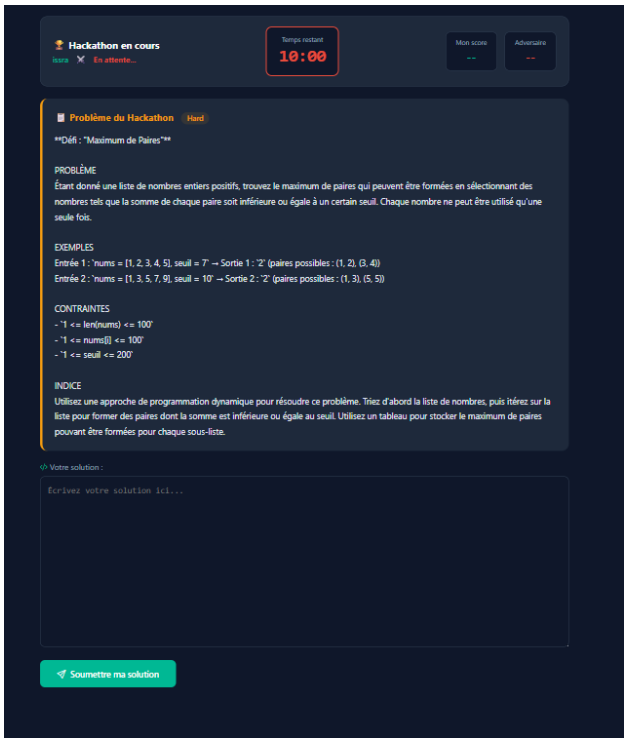


Figure 5.9– Interface de l'arène du hackathon

Interface résultat final hackathon : La figure 5.10 présente le modal de résultat final avec l'emoji de victoire/défaite, la comparaison des scores, et l'évaluation comparative IA des deux solutions (complexité et optimisation).

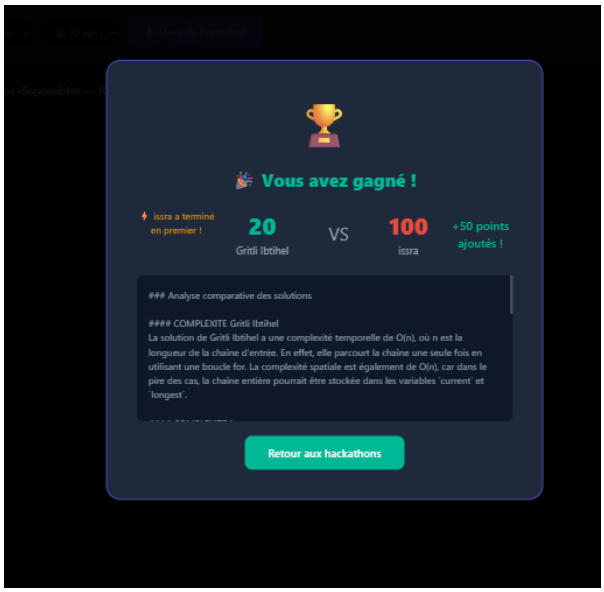


Figure 5.10– Interface de modal de résultat final

5.4 Conclusion :

-La cinquième et dernière release constitue l'aboutissement du projet Codify. Le Sprint 5 a permis de réaliser l'ensemble des fonctionnalités gamifiées décrites dans le backlog produit : exercices adaptatifs IA, système de points différencié, test de validation avec changement de niveau, hackathons en temps réel avec synchronisation des timers via WebSocket STOMP. L'intégration de WebSocket pour la synchronisation en temps réel des deux joueurs représente la fonctionnalité la plus complexe techniquement du projet.

Conclusion générale et Perspectives

→Ce projet nous a permis de développer une plateforme complète d'apprentissage de la programmation assistée par IA, en appliquant la méthodologie SCRUM sur 5 sprints.

Les principales réalisations sont :

- Un système d'authentification avec 3 niveaux adaptatifs
- Un éditeur de code avec analyse, optimisation et transformation IA
- Un système d'exercices générés dynamiquement par l'IA
- Un système de hackathons en temps réel via WebSocket
- Un tableau de bord avec historique et classement
- Un panneau administrateur complet

Perspectives d'amélioration :

- Ajout d'un vrai compilateur (Judge0 API)
- Application mobile (React Native)
- Système de badges et récompenses
- Mode équipe pour les hackathons
- Support de plus de langages (C++, JavaScript, PHP)